

[Notas video 4K4 20200519 Testing de Caja Negra Parte 1 y 4K4 20200519 Testing de Caja Negra Parte 2:](#)

[Notas video https://youtu.be/Ti5nwPeoyW0?si=wDJdI2LI-FktQkp__](https://youtu.be/Ti5nwPeoyW0?si=wDJdI2LI-FktQkp__) :

[Notas del Libro Beck Ken test-driven-development-by-example \(1\).pdf:](#)

[CAPÍTULO 28: Patrones de Desarrollo Guiado por Pruebas o ...
Test-Driven Development](#)

[CAPÍTULO 29: Patrones de Barra Roja](#)

[CAPÍTULO 30: Patrones de Pruebas](#)

[CAPÍTULO 31: Patrones de Barra Verde.](#)

[Diseño Ágil con TDD: disenAgilConTDD.pdf](#)

[Capítulo 2: ¿Qué es el Desarrollo Dirigido por Tests? \(TDD\)](#)

[Capítulo 3: Desarrollo Dirigido por Tests de Aceptación \(ATDD\)](#)

[Capítulo 4: Tipos de test y su importancia.](#)

[Características de un buen test Unitario \(F.I.R.S.T\)](#)

Notas video  4K4 20200519 Testing de Caja Negra Parte 1 y

 4K4 20200519 Testing de Caja Negra Parte 2 :

¿Qué es el testing?

Es un proceso destructivo, cuyo objetivo es encontrar defectos. Poder asumir a priori que una porción del SW para poder resolverlos posteriormente y de esa forma poder contribuir a la calidad del producto.

Asumimos como axioma que Existen defectos

Cuanto testing es suficiente??

Criterios:

good enough → Mientras probamos la funcionalidad con el transcurso del tiempo los errores disminuyen o también cuando hayamos consumido el criterio económico destinado a pruebas.

En metodologías ágiles hacen énfasis en automatización de las pruebas para poder repetir (pruebas de regresión) y economizar.

Por cada defecto corregido se puede incorporar otro defecto, por esto se deben de repetir los ciclos de pruebas una vez que se introducen correcciones.

Error vs Defecto

Si encontramos esta FALLA en la misma etapa en la que se originó se trata de un ERROR, pero si trasciende de la etapa de donde se originó es un DEFECTO.

El testing busca buscar DEFECTOS, no errores no es su objetivo como disciplina.

El testing unitario si encuentra errores, pero como se dijo. El testing no busca errores.

La idea es encontrar los errores ya que son más baratos de solucionar en el sentido económico pero también teniendo en cuenta la satisfacción del cliente. También solucionar errores representan menor esfuerzo que los defectos.

Si bien no importa que tan bien esté capacitado nuestro equipo, que herramientas y cuantas usemos ERRAR ES HUMANO. Va a haber una proporción que se conviertan en defectos.

El proceso de Testing consiste en 4 etapas:

Planificación y Control → Analisi y Diseño → Ejecución → Evaluación y Reportes

En el Análisis y Diseño se trabaja con los casos de prueba que son el artefacto por excelencia del TESTING como disciplina y que sirvan como guía para la ejecución de estas pruebas.

Niveles de Testing

1-Testing Unitario(Desarrollador): su objetivo a diferencia de su axioma es proveer de un mecanismo de la funcionalidad que estoy implementando, me permite verificar en lo que estoy trabajando.

2- Testing de Integración(Tester): Aca si buscamos encontrar defectos, buscamos encontrar las interfaces, boundaries y fronteras entre sistemas de gran o pequeños niveles. Buscar defectos en la unión de diferentes componentes representadas pro sus distintas interfaces,

Estamos buscando probar las integraciones de los distintos componentes.

Técnicas:

- Sandwich

Combina las estrategias Top-Down y Bottom-Up.

Se prueban simultáneamente los módulos superiores e inferiores, encontrándose en el medio.

El sistema se divide en tres capas:

Capa superior → integración descendente

Capa inferior → integración ascendente

Capa media → punto de encuentro

- Bottom-Up Integration (De abajo hacia arriba)

Se empieza por los **módulos de nivel inferior** (los que no dependen de otros).

Los módulos superiores se van agregando progresivamente.

Para simular los módulos aún no desarrollados, se usan **drivers** (programas temporales que llaman a los módulos inferiores).

Ventajas:

Los módulos de bajo nivel se prueban completamente.

No requiere stubs, solo drivers.

Errores en componentes básicos se detectan temprano.

Desventajas:

No se prueba el flujo de control global hasta el final.

Requiere muchos drivers.

No se obtiene una versión funcional del sistema hasta etapas avanzadas.

- Top-Down Integration (De arriba hacia abajo)

Se comienza por los **módulos de más alto nivel** (los que controlan la aplicación o llaman a otros módulos).

Los módulos inferiores aún no desarrollados se reemplazan con **stubs** (simulaciones temporales).

Ventajas:

Se prueban las funciones principales del sistema primero.

Facilita la detección de errores en la lógica de control.

Se tiene un prototipo funcional rápidamente.

Desventajas:

Los módulos de bajo nivel se prueban tarde.

La creación de stubs puede ser costosa o compleja.

No se pueden observar completamente las salidas finales hasta integrar todo.

3-Testing de Sistema(Tester): ya acá se prueba una funcionalidad en su totalidad, lo que se hace es ejecutar cierto camino con determinadas características para ver si el resultado recibido sea parecido al esperado.

Es encontrar defectos que surgieran de la implementación, teniendo en cuenta que para ejecutar testing de sistema debemos tener la funcionalidad implementada y por esto si o si encontramos DEFECTOS.

4-Tester de Aceptación(PO, usuario o cliente; la persona que finalmente nos va a dar la aceptación): Momento de lo que estamos entregando corresponde al requerimiento planteado o la expectativa no manifiesta.

Estrategias:

Aplicar un criterio económico maximizando la cantidad de testing minimizando el esfuerzo.

De aquí surgen los Casos de Prueba, son los guías del testing que contiene un conjunto de condiciones o pasos que se deben ejecutar para que el resultado esperado sea o no igual que el resultado obtenido.

Buscamos diseñar casos de prueba de forma inteligente y de forma económica para maximizar la cantidad de defectos encontrados.

En la estrategia de testing de caja blanca disponemos del código para poder ver el detalle disponiendo del código para diseñar el caso de prueba disponiendo de su cobertura.

En cambio en testing de caja negra no cuento con la estructura interna del software, solo analizo las inputs y outputs que me da el sistema.

Métodos:

¿Para que los usamos? El tiempo y el presupuesto es limitado.

Hay que pasar por la mayor cantidad de funcionalidades con la menor cantidad de pruebas.

Caja Negra

- Basado en especificaciones.
 - Particiones de equivalencia: es un conjunto de valores que puede tomar una condición externa el cual si tomo cualquier valor el resultado de la ejecución de la función es EQUIVALENTE.
- ¿Qué hace? analiza primero cuales son las diferentes condiciones externas que van a estar involucradas en la funcionalidad (ENTRANTES Y SALIENTES)
- Una vez que identifique estas condiciones externas que participan para cada condición externa voy a analizar los subconjuntos de valores posibles que pueden tomar cada una de estas que producen un resultado equivalente
- Requiere dos pasos:
1. identificar las clases de equivalencia (Válidas y no Válidas)

Estas las podemos dividir en subconjuntos de variables que puede tomar la condición que van a producir el mismo resultado. Podemos identificar:

 - rango de valores continuos.
 - valores discretos.
 - selección simple.
 - selección múltiple.
 2. Identificar los casos de prueba: elijo una clase de equivalencia y armó el paso a paso(una receta), para esto me sirven las clases de equivalencia para poder recrear con los paso a paso el resultado de un subconjunto que obtendrá la misma salida
- Análisis de valores límites.: Es una variante de la partición de equivalencia, en vez de seleccionar cualquier elemento como representativo de una clase de equivalencia, se seleccionan los bordes de una clase.
- Basado en la experiencia.
 - adivinanza de Defectos
 - testing exploratorio

Notas video <https://youtu.be/Ti5nwPeoyW0?si=wDJdI2LI-FktQkp> :

Conocemos la entrada y la salida pero no la estructura interna, vemos cómo se comporta el sistema viendo las salidas a partir de un conjunto de variables de entrada.

Es difícil hacer una cobertura completa al sistema, si este es grande se me hará imposible abarcar todas sus funcionalidades.

Métodos que aparecen para encontrar defectos sin gastar muchos recursos

Clases de equivalencia y Límites.

Caso de prueba: Definir un escenario donde probamos esas clases de equivalencia y definimos sus resultados que debe de otorgar el sistema que se deben de contrastar con los resultados esperados del sistema

Para qué sirve, probar con datos representativos de esa clase de equivalencia para mejorar y disminuir el esfuerzo de pruebas que van a arrojar un resultado equivalente.

Método de probar los valores límites, probar situaciones que son borde porque estadísticamente es donde suceden los errores.

Por ejemplo, al realizar la compra de bebidas alcohólicas, el resultado para todos los menores de 18 años debe de ser el mismo y lo mismo para los mayores de 18. Entonces no tiene sentido hacer pruebas en los rangos intermedios si podemos hacerlo con los límites y el resultado de estos debe de ser el mismo para el resto de casos con la misma condición.

Prioridad:

Alta --> caminos felices, son la funcionalidad core de SW. Son los que deben de salir bien

Media --> Son los casos alternativos que son importantes, por ejemplo que si en un sistema de pedidos se ingresa una tarjeta de crédito para realizar la compra y su formato es incorrecto y no se valida no debe de poder funcionar. Se deben de probar si o si.

Baja --> validaciones de datos, cuanto menos moleste y frene el SW se le va a dar menos prioridad

Precondiciones que se deben de cumplir en el SW para que se realice el, son las condiciones que se deben de haber ejecutado antes para que se puedan ejecutar los pasos de la prueba

Los pasos se ingresa información actual que deben de ser secuencial con la información necesaria demostrando lo que se debe de ingresar y ser específico de lo que van a usar para que sea replicable

Todos pueden de ejecutarlo replicando y obteniendo el mismo resultado

Resultado esperado, que debe de mostrar, cálculos, mandar un mail o generar algo que el sistema debe de mostrar

A continuación te adjuntamos dos archivos, yo solo quiero que te centres en los casos de prueba y me digas si estan bien, me expliques los que ya tenemos solo por ahora.

Notas del Libro  Beck Ken test-driven-development-by-example (1).pdf :

CAPÍTULO 28: Patrones de Desarrollo Guiado por Pruebas o ... Test-Driven Development

¿Qué podemos entender por probar?

¿Cuándo probamos?

¿Cómo elegimos qué lógica probar?

¿Cómo elegimos qué datos probar?

Los patrones de *Test-Driven Development* (TDD) ayudan a mantener el flujo de desarrollo cuando las decisiones se vuelven confusas o inciertas. Funcionan como un conjunto de prácticas probadas para guiar el diseño y la implementación del código mediante pruebas automatizadas.

Test

Testing significa evaluar. No se debería de lanzar ni el más mínimo cambio sin probarlo.

¿Qué sucede cuando aumenta el nivel de estrés?

La Figura 1 tiene al Estrés conectado negativamente con las Pruebas, conectado negativamente con los Errores, conectado positivamente con el Estrés.

Este es un bucle de retroalimentación positiva. Cuanto más estrés sientes, menos pruebas harás. Cuantas menos pruebas hagas, más errores cometerás. Cuantos más errores cometes, más estrés sentirás. Enjuagar y repetir.

¿Cómo sales de tal bucle? Introduce un nuevo elemento, reemplaza uno de los elementos o cambia las flechas. En este caso, reemplazaremos "probar" con "pruebas automatizadas".

La Figura 2 tiene al Estrés conectado positivamente con las Pruebas Automatizadas, conectado negativamente con los Errores y el Estrés, y los Errores conectados positivamente con el Estrés.

"¿Acabo de romper algo más con ese cambio?" Con las pruebas automatizadas, cuando empiezo a sentir estrés, ejecuto las pruebas. Las pruebas son la Piedra Filosofal del Programador, transmutando el miedo en aburrimiento. "No, todas las pruebas siguen en verde". Cuanto más estrés siento, más ejecuto las pruebas. Ejecutar las pruebas me da inmediatamente una buena sensación y reduce la cantidad de errores que cometo, lo que reduce aún más el estrés que siento.

"¡No tenemos tiempo para ejecutar las pruebas. Solo lánzalo!" La segunda imagen no está garantizada. Si el nivel de estrés aumenta lo suficiente, se rompe. Sin embargo, con las pruebas automatizadas tienes la oportunidad de elegir tu nivel de miedo.

¿Deberías ejecutar la prueba después de escribirla, aunque sepas que va a fallar? No, no te molestes. Por ejemplo, estaba trabajando con un par de programadores jóvenes muy agudos implementando transacciones en memoria (una técnica genial que todos los lenguajes de programación deberían tener). ¿Cómo íbamos a implementar la reversión si iniciábamos una transacción, cambiábamos algunas variables y luego dejábamos que la transacción fuera recolectada como basura? Bastante simple de probar, jóvenes. Atrás y vean al maestro en el trabajo. Aquí está la prueba. Ahora, ¿cómo vamos a implementar esto?

Dos horas después, horas empañadas de frustración porque un error al implementar características de tan bajo nivel generalmente bloquea el entorno de desarrollo, volvimos al punto donde habíamos comenzado. Escribí la prueba. La ejecuté por capricho. Pasó. Duh...

El punto central del mecanismo de transacción era que las variables no cambiaban realmente hasta que la transacción se confirmaba. Está bien, supongo que podrías seguir adelante y ejecutar esa nueva prueba si quieres.

Prueba Aislada

¿Cómo debería afectarse la ejecución de las pruebas entre sí? En absoluto.

Cuando era un programador joven, hace mucho, mucho tiempo, cuando teníamos que desenterrar nuestros propios bits de la nieve y llevar pesados cubos de ellos descalzos de regreso a nuestros cubículos dejando pequeñas huellas sangrientas para que los lobos nos siguieran... Lo siento, solo estoy recordando. Mi primera experiencia con pruebas automatizadas fue tener un conjunto de pruebas de larga duración, nocturnas, basadas en GUI (ya sabes, grabar las pulsaciones de teclas y los eventos del mouse y reproducirlos) para un depurador en el que estaba trabajando (¡hola Jothy, hola John!). Cada mañana cuando llegaba, había una ordenada pila de papel en mi silla describiendo las ejecuciones de pruebas de la noche anterior (¡hola Al!). En los buenos días, había una sola hoja resumiendo que nada se rompió. En los malos días, había muchas, muchas hojas, una por cada prueba rota. Empecé a temer los días en que veía un montón de papel en mi silla.

Saqué dos lecciones de esta experiencia. Primero, hacer que las pruebas sean tan rápidas de ejecutar que pueda ejecutarlas yo mismo y ejecutarlas a menudo. De esa manera puedo detectar errores antes de que anyone else los vea, y no tengo que temer llegar por la mañana. Segundo, noté después de un tiempo que una gran pila de papel generalmente no significaba una gran lista de problemas. Más a menudo significaba que una prueba se había roto temprano, dejando el sistema en un estado impredecible para la siguiente prueba.

Intentamos solucionar este problema iniciando y deteniendo el sistema entre cada prueba, pero tomaba demasiado tiempo, lo que me enseñó otra lección sobre buscar pruebas a una escala más pequeña que la de toda la aplicación. Pero la lección principal que aprendí fue que las pruebas deberían poder ignorarse por completo. Si tenía una prueba rota, quería un problema. Si tenía dos pruebas rotas, quería dos problemas.

Una implicación conveniente de las pruebas aisladas es que las pruebas son independientes del orden. Si quiero tomar un subconjunto de pruebas y ejecutarlas, puedo hacerlo sin preocuparme de que una prueba se rompa ahora porque falta una prueba prerequisite.

El rendimiento es la razón habitual citada para que las pruebas compartan datos. Una segunda implicación de las pruebas aisladas es que tienes que trabajar, a veces mucho, para dividir tu problema en pequeñas dimensiones ortogonales, de modo que configurar el entorno para cada prueba sea fácil y rápido. Aislar pruebas te anima a componer soluciones a partir de muchos objetos altamente cohesivos y débilmente acoplados. Siempre escuché que esta era una buena idea, y me alegré cuando lo logré, pero nunca supe exactamente cómo lograr regularmente alta cohesión y bajo acoplamiento hasta que comencé a escribir pruebas aisladas.

Lista de Pruebas

¿Qué deberías probar? Antes de comenzar, escribe una lista de todas las pruebas que sabes que tendrás que escribir.

La primera parte de nuestra estrategia para lidiar con el estrés de la programación es nunca dar un paso hacia adelante a menos que sepamos dónde va a caer nuestro pie. Cuando nos sentamos a una sesión de programación, ¿qué es lo que pretendemos lograr?

Una estrategia para realizar un seguimiento de lo que estamos tratando de lograr es mantenerlo todo en nuestra cabeza. Probé esto durante varios años y descubrí que entraba en un bucle de retroalimentación positiva. Cuanta más experiencia acumulaba, más cosas sabía que podrían necesitar hacerse. Cuantas más cosas sabía que podrían necesitar hacerse, menos atención prestaba a lo que estaba haciendo. Cuanta menos atención prestaba a lo que estaba haciendo, menos lograba. Menos lograba, más cosas sabía que necesitaban hacerse.

Simplemente ignorar elementos aleatorios en la lista y programar al azar no pareció funcionar para romper este ciclo.

Adquirí el hábito de escribir todo lo que quería lograr en las próximas horas en un trozo de papel al lado de mi computadora. Tenía una lista similar, pero con un alcance semanal o mensual, clavada en la pared. Tan pronto como tenía todo eso escrito, sabía que no iba a olvidar algo. Cuando surgía un nuevo elemento, decidía rápida y conscientemente si pertenecía a la lista de "ahora", a la lista de "después", o realmente no necesitaba hacerse en absoluto.

Aplicado al desarrollo guiado por pruebas, lo que ponemos en la lista son las pruebas que queremos implementar. Primero, pon en la lista ejemplos de cada operación que sabes que necesitas implementar. Luego, para aquellas operaciones que aún no existen, pon la versión nula de esa operación en la lista. Finalmente, enumera todas las refactorizaciones que crees que tendrás que hacer para tener código limpio al final de esta sesión.

En lugar de esbozar las pruebas, podríamos simplemente implementarlas todas. Hay un par de razones por las que escribir pruebas en masa no ha funcionado para mí. Primero, cada prueba que implementas es un poco de inercia cuando tienes que refactorizar. Con herramientas de refactorización automatizada (por ejemplo, tienes un elemento de menú que cambia el nombre de la declaración y todos los usos de una variable) esto es menos problemático, pero cuando has implementado diez pruebas y luego descubres que los argumentos necesitan estar en el orden opuesto, es mucho menos probable que limpies. En segundo lugar, si tienes diez pruebas rotas, estás muy lejos de la barra verde. Si quieres llegar al verde rápidamente, tienes que tirar las diez pruebas. Si quieres que todas las pruebas funcionen, vas a estar mirando una barra roja durante mucho tiempo. Si eres lo suficientemente adicto a la barra verde y no puedes ir al baño si la barra está roja, eso puede ser una eternidad.

Los montañeros conservadores tienen una regla: de tus cuatro manos y pies, tres de ellos deben estar sujetos en todo momento. Los movimientos dinámicos donde sueltas dos a la

vez son mucho más peligrosos. La forma pura de TDD, donde nunca estás a más de un cambio de distancia de una barra verde, es como esa regla de tres de cuatro.

A medida que haces que las pruebas se ejecuten, la implementación implicará nuevas pruebas. Escribe las nuevas pruebas en la lista. Lo mismo con las refactorizaciones. "Esto se está poniendo feo." "<suspiro> Ponlo en la lista. Lo haremos antes de que verifiquemos".

Los elementos que quedan en la lista cuando la sesión termina deben ser atendidos. Si realmente estás a mitad de camino de una funcionalidad, usa la misma lista más tarde. Si has descubierto refactorizaciones más grandes que están fuera del alcance por el momento, muévelas a la lista de "después". No recuerdo haber movido nunca un caso de prueba a la lista de "después". Si puedo pensar en una prueba que podría no funcionar, hacer que funcione es más importante que lanzar mi código.

Prueba Primero

¿Cuándo deberías escribir tus pruebas? Antes de escribir el código que se va a probar.

No probarás después. Tu objetivo como programador es la funcionalidad en ejecución. Sin embargo, necesitas una forma de pensar en el diseño, necesitas un método para el control del alcance.

Veamos el diagrama de influencia habitual que relaciona el estrés y las pruebas (pero no las pruebas de estrés, eso es diferente):

Estrés arriba conectado negativamente con pruebas abajo conectado negativamente con estrés.

Cuanto más estrés sientes, menos probable es que pruebes lo suficiente. Cuando sabes que no has probado lo suficiente, añades a tu estrés. Bucle de retroalimentación positiva. Una vez más, necesita haber una manera de romper el bucle.

¿Y si adoptáramos la regla de que siempre probaríamos primero? Entonces podemos invertir el diagrama y obtener un ciclo virtuoso:

Prueba-primero arriba conectado negativamente con estrés abajo conectado negativamente con Prueba-Primero...

Cuando probamos primero, reducimos el estrés, lo que hace que sea más probable que probemos. Sin embargo, hay muchos otros elementos que alimentan el estrés, por lo que las pruebas deben vivir en otros ciclos virtuosos o serán abandonadas cuando el estrés aumente lo suficiente.

Sin embargo, el beneficio inmediato de las pruebas (una herramienta de diseño y control de alcance) sugiere que podremos comenzar a hacerlo y seguir haciéndolo incluso bajo estrés moderado.

Afirmar Primero

¿Cuándo deberías escribir las afirmaciones? Intenta escribirlas primero.

¿No te encanta la auto-similitud?

¿Dónde deberías comenzar a construir un sistema? Con las historias que quieres poder contar sobre el sistema terminado.

¿Dónde deberías comenzar a escribir un poco de funcionalidad? Con las pruebas que quieres que pasen con el código terminado.

¿Dónde deberías comenzar a escribir una prueba? Con las afirmaciones que pasarán cuando esté terminada.

Jim Newkirk me presentó esta técnica. Cuando pruebo afirmar-primero, encuentro que tiene un poderoso efecto simplificador. Cuando estás escribiendo una prueba, estás resolviendo varios problemas a la vez, incluso si ya no tienes que pensar en la implementación.

¿Dónde pertenece la funcionalidad? ¿Es una modificación de un método existente, un nuevo método en una clase existente, un nombre de método existente implementado en un nuevo lugar o una nueva clase?

¿Cómo se deben llamar los nombres?

¿Cómo vas a comprobar la respuesta correcta?

¿Cuál es la respuesta correcta?

¿Qué otras pruebas sugiere esta prueba?

Cerebros del tamaño de un guisante como el mío posiblemente no puedan hacer un buen trabajo resolviendo todos estos problemas a la vez. Los dos problemas de la lista que se pueden separar fácilmente del resto son "¿cuál es la respuesta correcta?" y "¿cómo voy a comprobarlo?"

Datos de Prueba

¿Qué datos usas para las pruebas de "prueba primero"? Usa datos que faciliten la lectura y el seguimiento de las pruebas.

Estás escribiendo pruebas para una audiencia. No esparzas valores de datos solo por esparcirlos. Si hay una diferencia en los datos, debería ser significativa. Si no hay una diferencia conceptual entre 1 y 2, usa 1.

Los Datos de Prueba no son una licencia para quedarse corto en confianza total. Si tu sistema tiene que manejar múltiples entradas, tus pruebas deberían reflejar múltiples entradas. Sin embargo, no tengas una lista de 10 elementos como datos de entrada si una lista de 3 elementos te llevará a las mismas decisiones de diseño e implementación.

Un truco en los Datos de Prueba es intentar nunca usar la misma constante para significar más de una cosa. Si estoy probando un método `plus()`, es tentador probar $2 + 2$, ya que ese es el ejemplo clásico de suma, o $1 + 1$, ya que es tan simple. ¿Qué pasaría si invertimos los argumentos en la implementación? (Está bien, está bien, eso no importa en el caso de `plus()`, pero ya entiendes la idea). Si usamos 2 para el primer argumento, deberíamos usar 3, por ejemplo, para el segundo. (" $3 + 4$ " fue un caso de prueba crucial al poner en marcha una nueva máquina virtual de Smalltalk en los viejos tiempos).

La alternativa a los Datos de Prueba son los Datos Realistas, donde usas datos del mundo real. Los Datos Realistas son útiles cuando:

Estás probando sistemas en tiempo real usando trazas de eventos externos recopiladas de la ejecución real.

Estás comparando la salida del sistema actual con la salida de un sistema anterior (Pruebas Paralelas).

Estás refactorizando una simulación y esperas exactamente las mismas respuestas cuando termines, particularmente si la precisión del punto flotante puede ser un problema.

Datos Evidentes

¿Cómo representas la intención de los datos? Incluye los resultados esperados y reales en la propia prueba e intenta que su relación sea aparente.

Estás escribiendo pruebas para un lector, no solo para la computadora. Alguien dentro de décadas se estará preguntando: "¿En qué diablos estaba pensando este tipo?" Te gustaría dejar tantas pistas como sea posible, especialmente si ese lector frustrado vas a ser tú.

Un efecto secundario beneficioso de los Datos Evidentes es que facilita la programación. Una vez que hemos escrito la expresión en la afirmación, sabemos lo que necesitamos programar. De alguna manera tenemos que hacer que el programa evalúe una división y una multiplicación. Incluso podemos usar "Fingirlo" para descubrir dónde pertenecen las operaciones de forma incremental.

Los Datos Evidentes parecen ser una excepción a la regla de que no quieres números mágicos en tu código. Dentro del alcance de un solo método, la relación entre los 5 es obvia. Sin embargo, si tuviera constantes simbólicas que ya estuvieran definidas, usaría la forma simbólica.

CAPÍTULO 29: Patrones de Barra Roja

Estos patrones tratan sobre cuándo escribir pruebas, donde escribirlas y cuándo dejar de escribirlas.

Pruebas de un paso

¿Qué pruebas deberá elegir a continuación de la lista? Elegí una prueba que te enseñe algo y de la que estés seguro de que puedes implementarla.

Cada prueba debe representar un paso hacia tu objetivo general. Si estamos mirando la siguiente Lista de Pruebas, ¿Cuáles deberíamos elegir después?

Plus (Sumar)

Minus (Restar)

Times (Multiplicar)

Divide (Dividir)

Plus like (Sumar similares)

Equals (Igualar)

Equals null (Igualar a nulo)

Null exchange (Intercambio nulo)

Exchange one currency (Intercambiar una moneda)

Exchange two currencies (Intercambiar dos monedas)

Cross rate (Tasa cruzada)

No hay una respuesta correcta. Lo que para mí es un paso, sin haber implementado nunca estos objetos antes, será una décima parte de un paso para ti, con tu vasta experiencia.

Cuando miro una Lista de Pruebas, pienso: "Eso es obvio, eso es obvio, no tengo idea, obvio, ¿en qué estaba pensando con esa?, ah, esta puedo hacerla". Esa última prueba es la que implemento a continuación. No me pareció obvia, pero también estoy seguro de que puedo hacerla funcionar.

Si no encuentras ninguna prueba en la lista que represente un paso, añade algunas pruebas nuevas que representen un progreso hacia los elementos que allí se encuentran.

Un programa creado a partir de pruebas como esta puede parecer escrito de arriba hacia abajo (top-down), porque puedes comenzar con una prueba que representa un caso simple de toda la computación. Un programa creado a partir de pruebas también puede parecer escrito de abajo hacia arriba (bottom-up), porque comienzas con piezas pequeñas y las agregas cada vez más grandes.

Ni "top-down" ni "bottom-up" describen realmente el proceso de manera útil. En primer lugar, una metáfora vertical es una visualización simplista de cómo los programas cambian con el tiempo. "Crecimiento" implica una especie de bucle de retroalimentación autosimilar donde el entorno afecta al programa y el programa afecta al entorno. En segundo lugar, si tenemos que tener una dirección en nuestra metáfora, "de lo conocido a lo desconocido" es una descripción útil. "Conocido-a-desconocido" implica que tenemos algún conocimiento y experiencia para aprovechar, y que esperamos aprender en el transcurso del desarrollo. Une estos dos conceptos y tenemos programas que crecen de lo conocido a lo desconocido.

Prueba de Inicio

¿Con qué prueba deberías empezar? Comienza probando una variante de una operación que no haga nada.

La primera pregunta que debes hacerte con una nueva operación es "¿Dónde pertenece?" Hasta que no hayas respondido a esta pregunta, no sabes qué escribir para la prueba. En el espíritu de resolver un problema a la vez, ¿cómo podemos responder solo a esta pregunta y a ninguna otra?

Si escribes primero una prueba "realista", te encontrarás resolviendo un montón de problemas a la vez:

- * ¿Dónde pertenece la operación?
- * ¿Cuáles son las entradas correctas?
- * ¿Cuál es la salida correcta dadas esas entradas?

Comenzar con una prueba realista te dejará demasiado tiempo sin retroalimentación.

Rojo/verde/refactor, rojo/verde/refactor. Quieres que ese ciclo sea de minutos.

Puedes acortar el ciclo eligiendo entradas y salidas que sean trivialmente fáciles de descubrir. Por ejemplo, alguien publicó en el grupo de noticias de Extreme Programming preguntando sobre cómo escribir una prueba primero para un reductor de polígonos. La entrada es una malla de polígonos y la salida es una malla de polígonos que describe precisamente la misma superficie, pero con el menor número posible de polígonos. "¿Cómo puedo conducir con pruebas este problema si hacer que una prueba funcione requiere leer tesis doctorales?"

La Prueba de Inicio proporciona una respuesta:

- * La salida debe ser la misma que la entrada. Algunas configuraciones de polígonos ya están normalizadas, incapaces de una mayor reducción.
- * La entrada debe ser lo más pequeña posible, como un solo polígono, o incluso una lista vacía de polígonos.

Mi Prueba de Inicio fue así:

```
```java
Reducer r = new Reducer(new Polygon());
assertEquals(0, reducer.result().npoints);
```
```

¡Bing! La primera prueba está funcionando. Ahora, por todas las demás pruebas de la lista... Se aplica la Prueba de un Paso. Elige una Prueba de Inicio que te enseñe algo pero de la que estés seguro de que puedes hacerla funcionar rápidamente. Si estás implementando algo por enésima vez, elige una prueba que requiera una o dos operaciones. Estarás justificadamente seguro de que puedes hacerla funcionar. Si estás implementando algo complejo y complicado por primera vez, necesitas una pequeña píldora de valor inmediatamente.

Encuentro que mi Prueba de Inicio a menudo está en un nivel más alto, más como una prueba de aplicación, que las pruebas siguientes. Un ejemplo que suelo desarrollar con pruebas es un servidor simple basado en sockets. La primera prueba es así:

```
```
StartServer
Socket = new Socket
Message = "hello"
Socket.write(message)
AssertEquals(message, socket.read)
```
```

El resto de las pruebas se escriben en el servidor solo, "asumiendo que recibimos una cadena como esta..."

Otra Prueba de Inicio surgió del juego de la Vida. La gente que lo discutía hablaba sobre pruebas para celdas, tamaños, y arrays bidimensionales frente a tablas hash con puntos como claves. Mi Prueba de Inicio fue así:

Prueba de Explicación

¿Cómo difundes el uso de las pruebas automatizadas? Pide y da explicaciones en términos de pruebas.

Puede ser frustrante ser el único que usa TDD en un equipo. Sin embargo, pronto notarás menos problemas de integración y menos reportes de defectos en el código probado, y los diseños serán más simples y fáciles de explicar. Incluso ha sucedido que la gente se entusiasma mucho con las pruebas, y con probar primero.

Cuidado con el entusiasmo de los recién convertidos. Nada detendrá la propagación del TDD más rápido que imponerlo en la cara de las personas. Si eres un gerente o líder, no puedes obligar a nadie a cambiar su forma de trabajar.

¿Qué puedes hacer? Una forma simple de empezar es comenzar a pedir explicaciones en términos de casos de prueba. "Déjame ver si entiendo lo que dices. Por ejemplo, si tengo un Foo como este y un Bar como ese, ¿la respuesta debería ser 76?" Una técnica complementaria es comenzar a dar explicaciones en términos de pruebas. "Así es como funciona ahora. Cuando tengo un Foo como este y un Bar como ese, la respuesta es 76. Sin embargo, si tengo un Foo como ese y un Bar como este, me gustaría que la respuesta fuera 67."

Puedes hacer esto en niveles de abstracción más altos. Si alguien te está explicando un diagrama de secuencia, puedes pedir permiso para convertirlo a una notación más familiar. Luego, escribes un caso de prueba que contenga todos los objetos y mensajes externamente visibles del diagrama.

Prueba de Aprendizaje

¿Cuándo escribes pruebas para software producido externamente? Antes de la primera vez que vayas a usar una nueva funcionalidad del paquete.

Digamos que vamos a desarrollar algo sobre la biblioteca MIDP (Perfil de Información de Dispositivo Móvil) para Java. Queremos almacenar algunos datos en el RecordStore y recuperarlos. ¿Simplemente escribimos el código y esperamos que funcione? Esa es una forma de desarrollar.

Una alternativa es notar que estamos a punto de usar un nuevo método de una nueva clase. En lugar de solo usarlo, escribimos una pequeña prueba que verifica que la API funcione como se espera. Entonces, podríamos escribir:

```
```java
RecordStore store;

public void setUp() {
 store = RecordStore.openRecordStore("testing", true);
}

public void tearDown() {
 RecordStore.deleteRecordStore("testing");
}

public void testStore() {
 int id = store.addRecord(new byte[] {5, 6}, 0, 2);
 assertEquals(2, store.getRecordSize(id));
}
```

```
byte[] buffer = new byte[2];
assertEquals(2, store.getRecord(id, buffer, 0));
assertEquals(5, buffer[0]);
assertEquals(6, buffer[1]);
}
...
```

Si nuestro entendimiento de la API es correcto, la prueba pasará la primera vez. Jim Newkirk reportó un proyecto en el que las Pruebas de Aprendizaje se escribían rutinariamente. Cuando llegaban nuevas versiones del paquete, primero se ejecutaban las pruebas (y se arreglaban si era necesario). Si las pruebas no funcionaban, no tenía sentido ejecutar la aplicación porque ciertamente no funcionará. Una vez que las pruebas funcionaban, la aplicación funcionaba siempre.

#### Otra Prueba

¿Cómo evitas que una discusión técnica se desvíe del tema? Cuando surge una idea tangencial, añádela a la lista de pruebas y vuelve al tema.

Me encantan las discusiones errantes (ya has leído la mayor parte del libro, así que probablemente hayas llegado a esa conclusión tú mismo). Mantener una conversación estrictamente en curso es una gran manera de sofocar ideas brillantes. Saltas de aquí para allá, y ¿cómo llegamos aquí? ¿A quién le importa? ¡Esto es genial!

A veces la programación depende de los avances. Sin embargo, la mayoría de la programación es un poco más pedestar. Tengo diez cosas que implementar. Me convierto en un procrastinador consumado con el punto número cuatro. Refugiarme en conversaciones errantes es una de mis formas de evitar el trabajo (y quizás el miedo que lo acompaña).

Días enteros improductivos me han enseñado que a veces es mejor mantenerse en el camino. Cuando me siento así, las nuevas ideas son recibidas con respeto, pero no se les permite desviar mi atención. Las anoto en la lista y vuelvo a lo que estaba trabajando.

#### **Prueba de Regresión**

¿Qué es lo primero que haces cuando se reporta un defecto? Escribe la prueba más pequeña posible que falle, y que una vez que funcione, el defecto esté reparado.

Las pruebas de regresión son pruebas que, con previsión perfecta, habrías escrito al codificar originalmente. Cada vez que tienes que escribir una prueba de regresión, piensa en cómo podrías haber sabido escribir la prueba en primer lugar.

También obtendrás valor probando a nivel de toda la aplicación. Las pruebas de regresión para la aplicación les dan a tus usuarios la oportunidad de hablar concretamente sobre lo que está mal y lo que esperan. Las pruebas de regresión a menor escala son una forma de mejorar tus pruebas. El reporte de defectos será sobre un número negativo grande y extraño en un informe. La lección para ti es que necesitas probar el desbordamiento de enteros cuando escribes tu lista de pruebas.

Es posible que tengas que refactorizar el sistema antes de poder aislar fácilmente el defecto. El defecto en este caso era la forma en que tu sistema te decía: "Aún no has terminado de diseñar".

#### Pausa

¿Qué haces cuando te sientes cansado o estancado? Toma un descanso.

Toma una bebida, da un paseo, toma una siesta. Lávate las manos de tu compromiso emocional con las decisiones que acabas de tomar y los caracteres que escribiste.

A menudo, esta cantidad de distancia es todo lo que se necesita para liberar la idea que te ha estado faltando. Estarás levantándote cuando te des cuenta: "¡No lo he intentado con los parámetros invertidos!" Toma el descanso de todos modos. Date un par de minutos. La idea no desaparecerá.

Si no se te ocurre "la idea", revisa tus objetivos para la sesión. ¿Siguen siendo realistas o deberías elegir nuevos objetivos? ¿Es imposible lo que intentabas lograr? Si es así, ¿cuáles son las implicaciones para el equipo?

Dave Ungar llama a esto su Metodología de la Ducha. Si sabes qué escribir, escribe. Si no sabes qué escribir, toma una ducha. Muchos equipos serían más felices, más productivos y olerían mucho mejor si siguieran su consejo.

TDD es un refinamiento de la Metodología de la Ducha de Ungar. Si no sabes qué escribir, finjelo. Si el diseño "correcto" aún no está claro, triangula. Si aún no sabes qué escribir, entonces puedes tomar esa ducha.

Aquí hay un diagrama de influencia que muestra el bucle de retroalimentación positiva en funcionamiento:

\*La fatiga afecta negativamente el juicio, lo que afecta negativamente la fatiga.\*

Te estás cansando, por lo que eres menos capaz de darte cuenta de que estás cansado, así que continúas y te cansas más.

La salida de este bucle es introducir un elemento externo adicional.

\* A escala de horas, mantén una botella de agua junto a tu teclado para que la biología proporcione la motivación para descansos regulares.

\* A escala de un día, los compromisos después del horario laboral regular pueden ayudarte a detenerte cuando necesitas dormir antes que progresar.

\* A escala de una semana, los compromisos de fin de semana ayudan a liberar tus pensamientos conscientes que consumen energía del trabajo. (Mi esposa jura que tengo mis mejores ideas el viernes por la noche).

A escala de un año, las políticas de vacaciones obligatorias te ayudan a refrescarte por completo. Los franceses hacen esto bien: dos semanas contiguas de vacaciones no son suficientes. Pasas la primera semana descomprimiendo y la segunda semana preparándote para volver al trabajo. Por lo tanto, tres semanas, o mejor cuatro, son necesarias para que seas más efectivo el resto del año.

Hay otra cara de tomar descansos. A veces, cuando te enfrentas a un problema difícil, lo que necesitas hacer es seguir adelante, superarlo. Sin embargo, la cultura de la programación está tan infectada de un espíritu machista de "arruinaré mi salud, alienaré a mi familia y me mataré si es necesario" que no me siento obligado a dar ningún consejo en este sentido. Si te encuentras adicto a la cafeína y sin progresar en absoluto, quizás no deberías tomar tantos descansos. Mientras tanto, da un paseo.

Volver a Empezar (Do Over)

¿Qué haces cuando te sientes perdido? Tira el código y comienza de nuevo.

Estás perdido. Has tomado el descanso, te has lavado las manos en el arroyo, has hecho sonar la campana del templo tibetano, y aún así estás perdido. El código que iba tan bien hace una hora ahora es un desastre, no puedes pensar en cómo hacer que el siguiente caso de prueba funcione, y se te han ocurrido 20 pruebas más que realmente deberías implementar.

Esto me ha pasado varias veces al escribir este libro. Conseguía enredar un poco el código. "Pero tengo que terminar el libro. Los niños se están muriendo de hambre y los cobradores de facturas están golpeando la puerta". Mi reacción instintiva era desenredarlo lo suficiente para seguir adelante. Después de una pausa para reflexionar, empezar de nuevo siempre



tenía más sentido. La única vez que presioné a pesar de todo, tuve que tirar 25 páginas del manuscrito porque se basaban en una decisión de programación obviamente estúpida.

Mi ejemplo favorito de "Volver a Empezar" es una historia que me contó Tim Mackinnon. Estaba entrevistando a alguien simplemente pidiéndole que programara en pareja con él durante una hora. Al final de la sesión, habían implementado varios casos de prueba nuevos y hecho una buena refactorización. Sin embargo, era el final del día y se sintieron cansados cuando terminaron, así que descartaron su trabajo.

Si programas en pareja, cambiar de compañero es una buena manera de motivar "Volver a Empezar" de manera productiva. Intentarás explicar el complicado desastre que has hecho durante unos minutos, cuando tu nuevo compañero, completamente desvinculado de los errores que has cometido, tomará suavemente el teclado y dirá: "Lamento mucho ser tan torpe, pero ¿y si empezáramos así...?"

Escritorio Barato, Silla Buena

¿Qué configuración física deberías usar para el desarrollo guiado por pruebas? Consigue una silla muy buena, escatimando en el resto del mobiliario si es necesario.

No puedes programar bien si te duele la espalda. Sin embargo, hay organizaciones que gastarán cien mil dólares al mes en un equipo pero no gastarán diez mil dólares en sillas decentes.

Mi solución es usar mesas plegables baratas y feas para mis computadoras, pero comprar las mejores sillas que pueda encontrar. Tengo mucho espacio en el escritorio, y puedo conseguirlo más fácilmente, y estoy fresco y listo para programar por la tarde y por la mañana.

Ponte cómodo cuando estés programando en pareja. Limpia la superficie del escritorio lo suficiente como para poder deslizar el teclado de un lado a otro. Cada compañero debe poder sentarse cómodamente directamente frente al teclado cuando estén conduciendo. Uno de mis trucos de coaching favoritos es acercarme por detrás a una pareja que está trabajando arduamente y deslizar suavemente el teclado para que quede cómodamente colocado para la persona que escribe.

Manfred Lange señala que la asignación cuidadosa de recursos también se aplica al hardware de la computadora. Obtén máquinas baratas/lentas/viejas para correo electrónico individual y navegación, y las máquinas más potentes posibles para el desarrollo compartido.

## **CAPÍTULO 30: Patrones de Pruebas**

### **Prueba Hija**

¿Cómo haces que se ejecute un caso de prueba que resulta ser demasiado grande?

Escribe una prueba más pequeña que represente la parte rota de la prueba más grande.

Haz que la prueba más pequeña funcione. Reintroduce la prueba más grande.

El ritmo rojo/verde/refactor es tan importante para el éxito continuo que cuando corres el riesgo de perderlo, vale la pena un esfuerzo extra para mantenerlo. Esto me sucede comúnmente cuando escribo una prueba que accidentalmente requiere varios cambios para funcionar. Incluso diez minutos con una barra roja me ponen nervioso.

Cuando escribo una prueba que es demasiado grande, primero trato de aprender la lección.

¿Por qué fue demasiado grande? ¿Qué podría haber hecho diferente para que fuera más pequeña? ¿Cómo me siento ahora mismo?

Una vez realizada la introspección metafísica, elimino la prueba ofensiva y comienzo de nuevo. "Bueno, hacer funcionar estas tres cosas a la vez fue demasiado. Sin embargo, si

tuviera A, B y C funcionando, hacer funcionar todo sería pan comido". A veces realmente elimino la prueba, a veces simplemente cambio el nombre para que comience con una "x" y no se ejecute. (¿Puedo contarte un secreto? A veces ni siquiera me molesto en eliminar la prueba ofensiva. Shhhhh... Vivo con dos, cuenta 'em, dos pruebas rotas durante un par de minutos mientras hago funcionar la prueba hija. Podría estar cometiendo un error al hacer esto. Dos pruebas rotas podrían ser fácilmente un vestigio de mis malos tiempos de "probar-al-final-si-es-que-lo-hago").

Pruébalo de ambas maneras tú mismo. Observa si te sientes diferente, programas diferente, cuando tienes dos pruebas rotas. Responde como corresponda.

Objeto Simulado (Mock Object)

¿Cómo pruebas un objeto que depende de un recurso costoso o complicado? Crea una versión falsa del recurso que responda con constantes.

Hay material para al menos un libro en los Objetos Simulados, pero esto servirá como introducción.

El ejemplo clásico es una base de datos. Las bases de datos tardan mucho en iniciarse, son difíciles de mantener limpias y, si están ubicadas en un servidor remoto, vinculan tus pruebas a una ubicación física en una red. La base de datos también es una fuente fértil de errores en el desarrollo.

La solución es no usar una base de datos real la mayor parte del tiempo. La mayoría de las pruebas se escriben en términos de un objeto que actúa como una base de datos, pero que en realidad solo reside en la memoria.

```
```java
public void testOrderLookup() {
    Database db = new MockDatabase();
    db.expectQuery("select order_no from Order where cust_no is 123");
    db.setResult(new String[] {"Order 2", "Order 3"});
    ...
}
```
```

Si el MockDatabase no recibe la consulta que espera, lanza una excepción. Si la consulta es correcta, devuelve algo que parece un conjunto de resultados construido a partir de cadenas constantes.

Otro valor de los mocks, además del rendimiento y la confiabilidad, es la legibilidad. Puedes leer la prueba de arriba de principio a fin. Si tienes una base de datos de prueba llena de datos realistas, cuando ves que una consulta debería haber dado 14 respuestas, no tienes idea de por qué 14 es la respuesta correcta.

Si quieres usar Objetos Simulados, no puedes almacenar fácilmente recursos costosos en variables globales (incluso si se disfrazan de Singletons). Si lo haces, tendrás que configurar la global para que sea un Objeto Simulado, ejecutar la prueba y asegurarte de restablecer la global cuando termines.

Ha habido veces en que me enfurecí con esta restricción. Massimo Arnoldi y yo estábamos trabajando en código que dependía de un conjunto de tipos de cambio almacenados en una variable global. Cada prueba necesitaba diferentes subconjuntos de los datos, y a veces necesitaban diferentes tipos de cambio. Después de un tiempo intentando que la variable global funcionara, decidimos una mañana (las decisiones de diseño valientes llegan con más frecuencia por la mañana para mí) simplemente pasar el Exchange a donde lo necesitáramos. Pensamos que tendríamos que modificar cientos de métodos. Al final,

añadimos un parámetro a diez o quince métodos y limpiamos otros aspectos del diseño en el proceso.

Los mocks te alentarán a considerar cuidadosamente la visibilidad de cada objeto, reduciendo el acoplamiento en tus diseños.

Los Objetos Simulados añaden un riesgo al proyecto: ¿qué pasa si el mock no se comporta como el objeto real? Puedes reducir este riesgo teniendo un conjunto de pruebas para el mock que también se puedan aplicar al objeto real cuando esté disponible.

Auto-Desviación (Self Shunt)

¿Cómo pruebas que un objeto se comunica correctamente con otro? Haz que el objeto bajo prueba se comunique con el caso de prueba en lugar de con el objeto que espera.

Supongamos que queríamos actualizar dinámicamente la barra verde en la interfaz de usuario de pruebas. Si pudiéramos conectar un objeto al `TestResult`, podría ser notificado cuando una prueba se ejecutara, cuando fallara, cuando un suite completo comenzara y terminara, etc. Cada vez que se nos notificara que se ejecutó una prueba, actualizaríamos la interfaz. Aquí hay una prueba para esto:

```
```python
def testNotification(self):
    result = TestResult()
    listener = ResultListener()
    result.addListener(listener)
    WasRun("testMethod").run(result)
    assert 1 == listener.count
...
```
```

La prueba necesita un objeto para contar el número de notificaciones:

```
```python
class ResultListener:
    def __init__(self):
        self.count = 0
    def startTest(self):
        self.count = self.count + 1
...
```
```

Pero espera. ¿Por qué necesitamos un objeto separado para el listener? Podemos usar el propio caso de prueba. El `TestCase` en sí se convierte en una especie de Objeto Simulado.

```
```python
def testNotification(self):
    self.count = 0
    result = TestResult()
    result.addListener(self)
    WasRun("testMethod").run(result)
    assert 1 == self.count

def startTest(self):
    self.count = self.count + 1
...
```
```

Las pruebas escritas con Auto-Desviación tienden a leerse mejor que las escritas sin ella.

La prueba de arriba es un buen ejemplo. El contador era 0, y luego fue 1. ¿Cómo llegó a ser 1? Alguien debe haber llamado a `startTest()`. ¿Cómo se llamó a `startTest()`? Debe suceder al ejecutar la prueba. Este es otro ejemplo de simetría: la segunda versión del

método de prueba tiene los dos valores para el contador en un solo lugar, mientras que en la primera versión el contador se establece en 0 en una clase y se espera que sea 1 en otra. Auto-Desviación puede requerir que uses Extraer Interfaz para obtener una interfaz que implementar. Tendrás que decidir si extraer la interfaz es más fácil o si probar la clase existente como una caja negra es más fácil. Sin embargo, he notado que las interfaces extraídas para desvíos tienden a obtener su tercera y subsiguientes implementaciones poco después.

Como resultado de usar Auto-Desviación, verás pruebas en Java implementando todo tipo de interfaces extrañas. En lenguajes con tipado optimista (dinámico), la clase de caso de prueba solo necesita implementar las operaciones que realmente se usan en la ejecución de la prueba. En Java, sin embargo, tienes que implementar todas las operaciones de la interfaz, incluso si la mayoría de las implementaciones están vacías, por lo que preferirías que las interfaces fueran lo más estrechas posible. Las implementaciones deberían devolver un valor razonable o lanzar una excepción, dependiendo de si quieres que se te notifique si se invoca una operación inesperada.

Cadena de Registro (Log String)

¿Cómo pruebas que la secuencia en la que se llaman los mensajes es correcta? Mantén un registro en una cadena y añádele cuando se llame a un mensaje.

El ejemplo de xUnit sirve. Tenemos un Método Plantilla del que esperamos que llame a `setUp()`, un método de prueba y `tearDown()`, en ese orden. Al implementar los métodos para registrar en una cadena que fueron llamados, la prueba se lee muy bien:

```
```python
def testTemplateMethod(self):
    test = WasRun("testMethod")
    result = TestResult()
    test.run(result)
    assert("setUp testMethod tearDown " == test.log)
...

```

Y la implementación también es simple:

```
```python
class WasRun:
 def setUp(self):
 self.log = "setUp "
 def testMethod(self):
 self.log = self.log + "testMethod "
 def tearDown(self):
 self.log = self.log + "tearDown "
...

```

Las Cadenas de Registro son particularmente útiles cuando estás implementando Observer y esperas que las notificaciones lleguen en un cierto orden. Si esperabas ciertas notificaciones pero no te importaba el orden, podrías mantener un conjunto de cadenas y usar comparación de conjuntos en la aserción.

Cadena de Registro funciona bien con Auto-Desviación. El caso de prueba implementa los métodos en la interfaz desviada añadiendo al registro y luego devolviendo valores razonables.

## **Muñeco de Pruebas de Choque (Crash Test Dummy)**

¿Cómo pruebas código de error que es poco probable que se invoque? Invócalo de todos modos con un objeto especial que lance una excepción en lugar de hacer un trabajo real. El código que no se prueba no funciona. Esta parece ser la suposición segura. Entonces, ¿qué hacer con todas esas condiciones de error extrañas? ¿Tienes que probarlas también? Solo si quieres que funcionen.

Digamos que queremos probar qué le sucede a nuestra aplicación cuando el sistema de archivos está lleno. Podríamos trabajar mucho para crear muchos archivos grandes y llenar el sistema de archivos, o podríamos fingirlo. "Fingirlo" no suena muy digno, ¿verdad? Lo simularemos.

Aquí está nuestro muñeco de pruebas de choque para un archivo:

```
```java
private class FullFile extends File {
    public FullFile(String path) {
        super(path);
    }
    public boolean createNewFile() throws IOException {
        throw new IOException();
    }
}
...
```
```

Ahora podemos escribir nuestra prueba de Excepción Esperada:

```
```java
public void testFileSystemError() {
    File f = new FullFile("foo");
    try {
        saveAs(f);
        fail();
    } catch (IOException e) {
    }
}
...
```
```

Un Muñeco de Pruebas de Choque es como un Objeto Simulado, excepto que no necesitas simular todo el objeto. Las clases internas anónimas de Java funcionan bien para sabotear justo el método correcto para simular el error que queremos ejercitar. Puedes anular solo el método que quieres, justo allí en tu caso de prueba, haciendo que el caso de prueba sea más fácil de leer:

```
```java
public void testFileSystemError() {
    File f = new File("foo") {
        public boolean createNewFile() throws IOException {
            throw new IOException();
        }
    };
    try {
        saveAs(f);
        fail();
    } catch (IOException e) {
    }
}
```
```

}  
...

### **Prueba Rota (Broken Test)**

¿Cómo terminas una sesión de programación cuando programas solo? Deja la última prueba rota.

Richard Gabriel me enseñó el truco de terminar una sesión de escritura a mitad de una oración. Cuando te sientas de nuevo, miras la media oración y tienes que averiguar en qué estabas pensando cuando la escribiste. Una vez que recuperas el hilo del pensamiento, terminas la oración y continúas. Sin el impulso de terminar la oración, puedes pasar muchos minutos primero husmeando en qué trabajar a continuación, luego tratando de recordar tu estado mental, y finalmente volviendo a escribir.

Probé la técnica análoga para mis proyectos en solitario y realmente me gusta el efecto.

Termina una sesión en solitario escribiendo un caso de prueba y ejecutándolo para asegurarte de que no pasa. Cuando vuelvas al código, tienes un lugar obvio para comenzar, tienes un marcador concreto y obvio para ayudarte a recordar en qué estabas pensando, y hacer que esa prueba funcione debería ser un trabajo rápido, por lo que rápidamente volverás a ponerte en el camino de la victoria.

Pensé que me molestaría tener una prueba rota durante la noche. No es así, creo porque sé que el programa no está terminado. Una prueba rota no hace que el programa esté menos terminado, solo hace que el estado del programa sea manifiesto. La capacidad de retomar rápidamente un hilo de desarrollo después de semanas de pausa vale esa pequeña punzada de alejarse de una barra roja.

### **Verificación Limpia (Clean Check-in)**

¿Cómo terminas una sesión de programación cuando programas en equipo? Deja todas las pruebas funcionando.

"¿Me contradigo? Pues maldita sea."

--Bubba Whitman, el hermano estibador de Walt

Cuando eres responsable ante tus compañeros de equipo, la imagen cambia por completo. Cuando comienzas a programar en un proyecto de equipo, no sabes en detalle qué le ha pasado al código desde la última vez que lo viste. Necesitas comenzar desde un lugar de confianza y certeza. Por lo tanto, asegúrate siempre de que todas las pruebas estén funcionando antes de verificar tu código (un poco como cómo cada caso de prueba deja el mundo en un estado conocido-bueno, si eres propenso a las metáforas informáticas para el comportamiento humano, que yo no lo soy (usualmente)).

El conjunto de pruebas que ejecutas cuando verificas puede ser más extenso que el que ejecutas cada minuto durante el desarrollo (no renuncies a ejecutar todo el conjunto todo el tiempo hasta que sea lo suficientemente lento como para ser molesto). Ocasionalmente encontrarás una prueba rota en el conjunto de integración cuando intentes verificar. ¿Qué hacer?

La regla más simple es simplemente tirar tu trabajo y comenzar de nuevo. La prueba rota es una evidencia bastante fuerte de que no sabías lo suficiente para programar lo que acabas de programar. Si el equipo adoptara esta regla, habría una tendencia a que la gente verifique con más frecuencia porque la primera persona en verificar no arriesga perder ningún trabajo. Verificar con más frecuencia es probablemente algo bueno.

Un enfoque ligeramente más libertino es darte la oportunidad de arreglar el defecto e intentarlo de nuevo. Para evitar dominar los recursos de integración, probablemente

deberías rendirte después de unos minutos y comenzar de nuevo. Huelga decirlo, pero lo diré de todos modos, que comentar pruebas para que el conjunto pase está estrictamente prohibido, y es motivo para que tengas que pagar unas cervezas serias en la reunión de planificación fuera del sitio del viernes por la tarde.

## **CAPÍTULO 31: Patrones de Barra Verde.**

### **Fingirlo (Hasta que lo Logres)**

¿Cuál es tu primera implementación una vez que tienes una prueba rota? Devuelve una constante. Una vez que la prueba funcione, transforma gradualmente la constante en una expresión usando variables.

Un ejemplo simple ocurrió en nuestra implementación de xUnit.

```
python
return "1 run, 0 failed"
se convirtió en:
```

```
python
return "%d run, 0 failed" % self.runCount
y luego en:
```

```
python
return "%d run, %d failed" % (self.runCount, self.failureCount)
```

Fingirlo es un poco como clavar un pitón por encima de tu cabeza cuando escalas una roca. Realmente no has llegado allí todavía (la prueba está ahí pero la estructura del código es incorrecta). Sin embargo, cuando llegues allí, sabrás que estarás a salvo (la prueba seguirá ejecutándose).

Fingirlo realmente molesta a algunas personas. ¿Por qué harías algo que sabes que tendrás que eliminar? Porque tener algo funcionando es mejor que no tener nada funcionando, especialmente si tienes las pruebas para demostrarlo. Peter Hansen compartió esta historia:

"... algo sucedió ayer, donde, como dos novatos en TDD, mi compañero y yo nos apegamos agresivamente a la letra de la ley y cometimos pecados para hacer que una prueba funcionara rápidamente. En el proceso, nos dimos cuenta de que no habíamos implementado correctamente la prueba, así que volvimos y arreglamos eso, luego hicimos que el código funcionara nuevamente. El primer código funcional terminó no estando para nada a la vista cuando volvió a funcionar y nos miramos y dijimos: '¡Eh... mira eso!' porque ese enfoque nos había enseñado algo que no sabíamos".

¿Cómo una implementación falsa pudo haberles enseñado que su prueba estaba mal escrita? No lo sé, pero apuesto a que estaban contentos de no haber invertido en la solución real para descubrirlo.

Hay un par de efectos que hacen que Fingirlo sea poderoso:

Psicológico: Tener una barra verde se siente completamente diferente a tener una barra roja. Cuando la barra está verde, sabes en qué punto te encuentras. Puedes refactorizar desde allí con confianza.

Control de alcance: Los programadores son buenos para imaginar todo tipo de problemas futuros. Comenzar con un ejemplo concreto y generalizar a partir de ahí te impide confundirte prematuramente con preocupaciones externas. Puedes hacer un mejor trabajo resolviendo el problema inmediato porque estás concentrado. Cuando vas a implementar el siguiente caso de prueba, puedes concentrarte también en ese, sabiendo que la prueba anterior está garantizada que funciona.

¿Fingirlo viola la regla que dice que no escribes ningún código que no sea necesario? No lo creo, porque en el paso de refactorización estás eliminando la duplicación de datos entre el caso de prueba y el código. Cuando escribo<sup>1</sup>:

```
java
assertEquals(new MyDate("28.2.02"), new MyDate("1.3.02").yesterday());
```

<sup>1</sup> Gracias a Dierk König por el ejemplo.

```
java
public MyDate yesterday() {
 return new MyDate("28.2.02");
}
```

Hay duplicación entre la prueba y el código. Puedo cambiarlo escribiendo:

```
java
public MyDate yesterday() {
 return new MyDate(new MyDate("31.3.02").days()-1);
}
```

Pero todavía hay duplicación. Sin embargo, puedo eliminar la duplicación de datos (porque `this = MyDate("31.1.02")` para los propósitos de mi prueba) escribiendo:

```
java
public MyDate yesterday() {
 return new MyDate(this.days()-1);
}
```

No todo el mundo está convencido por este poco de sofistería, por lo que puedes usar la Triangulación, al menos hasta que te canses de ella y comiences a usar Fingirlo o incluso la Implementación Obvia.

Cuando uso Fingirlo, me recuerda a los largos viajes en coche con niños en el asiento trasero. Escribo la primera prueba, la hago funcionar de manera fea, y luego: "No me obligues a detener este coche y escribir otra prueba. Si tengo que detenerme, lo lamentarás". "Está bien, está bien, papá. Limpiaré el código. No tienes que ponerte tan nervioso".



## Triangulación

¿Cómo impulsas la abstracción de manera más conservadora con las pruebas? Solo abstraes cuando tienes dos o más ejemplos.

He aquí un ejemplo. Supongamos que queremos escribir una función que devuelva la suma de dos enteros. Escribimos:

```
java
public void testSum() {
 assertEquals(4, plus(3, 1));
}

private int plus(int augend, int addend) {
 return 4;
}
```

Si estamos triangulando hacia el diseño correcto, tenemos que escribir:

```
java
public void testSum() {
 assertEquals(4, plus(3, 1));
 assertEquals(7, plus(3,4));
}
```

Cuando tenemos el segundo ejemplo, podemos abstraer la implementación de plus():

```
java
private int plus(int augend, int addend) {
 return augend + addend;
}
```

La Triangulación es atractiva porque las reglas para ella parecen muy claras. Las reglas para Fingirlo, donde confiamos en nuestro sentido de duplicación entre el caso de prueba y la implementación falsa para impulsar la abstracción, parecen un poco vagas y sujetas a interpretación. Aunque parecen simples, las reglas para la triangulación crean un bucle infinito. Una vez que tenemos las dos afirmaciones y hemos abstraído la implementación correcta para plus, podemos eliminar una de las afirmaciones con el argumento de que es completamente redundante con la otra. Sin embargo, si hacemos eso, podemos simplificar la implementación de plus() para que solo devuelva una constante, lo que nos obliga a añadir una afirmación.

Solo uso la triangulación cuando estoy realmente, realmente inseguro sobre la abstracción correcta para el cálculo. De lo contrario, confío en la Implementación Obvia o en Fingirlo.

## Implementación Obvia

¿Cómo implementas operaciones simples? Simplemente impleméntalas.

Fingirlo y la Triangulación son pasos minúsculos. A veces estás seguro de que sabes cómo implementar una operación. Adelante. Por ejemplo, ¿realmente usaría Fingirlo para implementar algo tan simple como plus()? Por lo general, no. Simplemente escribiría la

implementación obvia. Si notara que me sorprenden las barras rojas, pasaría a pasos más pequeños.

No hay ninguna virtud particular en la naturaleza intermedia de Fingirlo y la Triangulación. Si sabes qué escribir y puedes hacerlo rápidamente, hazlo. Sin embargo, al usar solo la Implementación Obvia, te estás exigiendo perfección a ti mismo<sup>2</sup>. Psicológicamente, esto puede ser un movimiento devastador. ¿Qué pasa si lo que escribes no es realmente el cambio más simple que podría hacer que la prueba pase? ¿Qué pasa si tu compañero te muestra uno aún más simple? ¡Eres un fracaso! ¡Tu mundo se desmorona a tu alrededor! Te mueres. Te bloqueas.

Resolver "código limpio" al mismo tiempo que resuelves "que funcione" puede ser demasiado para hacerlo a la vez. Tan pronto como lo sea, vuelve a resolver "que funcione", luego "código limpio" con calma.

Lleva un registro de la frecuencia con la que te sorprenden las barras rojas al usar la Implementación Obvia. Me quedaré atrapado en estos ciclos donde escribiré una Implementación Obvia. No funcionará. Pero ahora estoy seguro de que sé lo que debería escribir, así que escribo eso. Tampoco funciona. Entonces ahora... Esto sucede especialmente con errores de "off-by-one" y errores positivo/negativo.

Quieres mantener ese ritmo rojo/verde/refactor. La Implementación Obvia es la segunda marcha. Prepárate para reducir la marcha si tu cerebro comienza a escribir cheques que tus dedos no pueden cobrar.

## Uno a Muchos

¿Cómo implementas una operación que funciona con colecciones de objetos? Implementarla sin colecciones primero, luego haz que funcione con colecciones.

Diseño Ágil con TDD:  [disenoAgilConTDD.pdf](#)

## Capítulo 2: ¿Qué es el Desarrollo Dirigido por Tests? (TDD)

### Objetivo del TDD

Es una técnica de diseño de SW que consiste en escribir pruebas automatizadas antes del código de producción. Su objetivo es:

- Implementar solo lo que el cliente necesita.
- Reducir defectos en producción.
- Crear código modular, reutilizable y preparado para el cambio.

### El algoritmo TDD: 3 pasos

Rojo: escribir una prueba que falle (específicamente).

Verde: Escribir el código mínimo para que la prueba pase.

Filosofía: Escribe el código más simple posible. Aunque parezca "trampa", esto nos fuerza a escribir más tests.

Refactorizar: Mejorar el código sin alterar su comportamiento.

Regla de oro: Solo refactorizamos cuando todos los tests están VERDE.

**Beneficios del TDD:**

- Mejorar la calidad del código.
- Facilitar el trabajo en equipo.
- Sirve como documentación viva del sistema.
- Ayuda a evitar el sobrediseño.

| Beneficio            | Explicación                                     |
|----------------------|-------------------------------------------------|
| Diseño Emergente     | La arquitectura surge naturalmente de los tests |
| Cobertura Automática | 100% de código probado por definición           |
| Documentación Viva   | Los tests muestran cómo usar el código          |
| Coraje para Cambiar  | Los tests dan confianza para refactorizar       |

**Consideraciones:**

No es solo para expertos: TDD es útil para todos los niveles.

No es magia: requiere práctica y disciplina.

Se puede aplicar en proyectos existentes aunque con más cuidado.

### Capítulo 3: Desarrollo Dirigido por Tests de Aceptación (ATDD)

**Problemas tradicional:**

- El cliente describe requisitos en documentos largos.
- Los desarrolladores interpretan de forma diferente.
- Surgen malentendidos que se descubren tarde.

### **Solución ATDD:**

Requisitos = ejemplos ejecutables.  
Validación continua con el cliente.

### **¿Qué es ATDD?**

Es una extensión de TDD a nivel de requisitos de negocio. Consiste en definir criterios de aceptación en forma de ejemplos ejecutables, escritos en colaboración con el cliente.

### **El Proceso ATDD Paso a Paso**

Paso 1: Escritura Colaborativa de Ejemplos  
Reunión con: Cliente, Desarrolladores, Testers  
Paso 2: Automatización con Frameworks  
Paso 3: Desarrollo Guiado por los Tests

### **Elementos claves:**

Historias de usuario: frases cortas que resumen un requisito.  
Testes de aceptación: Ejemplos concretos que validen que el sw cumple con lo esperado.  
Lenguaje del cliente: Se evita el lenguaje técnico.

### **Ventajas de ATDD:**

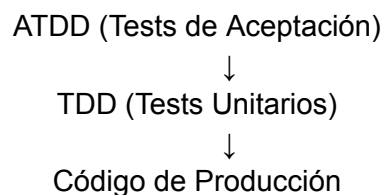
- claridad en lo que se debe construir.
- Reducción de malentendidos entre clientes y equipos.
- Validación automática de los criterios de aceptación.

### **Contexto necesario:**

ATDD funciona mejor en entornos ágiles, con colaboración estrecha entre clientes, analistas y desarrolladores.

### **Relación entre ATDD y TDD**

text



ATDD: "QUÉ hacer" (nivel negocio)

TDD: "CÓMO hacerlo" (nivel técnico)

### **Ventajas Clave de ATDD**

1. Alineación con el Cliente: Menos sorpresas al final
2. Especificaciones Ejecutables: Los requisitos no se quedan en papel
3. Regresión Automatizada: Si rompemos algo, los tests de aceptación fallan
4. Enfoque en Valor: Solo implementamos lo que el cliente realmente necesita

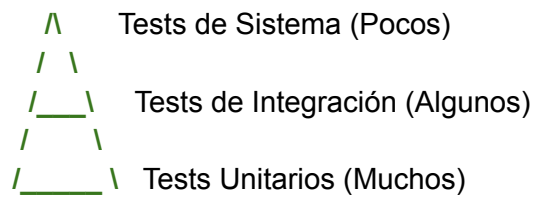
### **CONCLUSIÓN PRÁCTICA**

TDD te ayuda a diseñar mejor código desde lo pequeño.

ATDD asegura que estás construyendo lo correcto.  
La clasificación de tests te ayuda a mantener el balance adecuado.  
La combinación de estas prácticas resulta en:

- Software que funciona
- Código mantenible
- Cliente satisfecho
- Equipo productivo

#### Capítulo 4: Tipos de test y su importancia.



Ratio sugerido: 70% Unitarios - 20% Integración - 10% Sistema

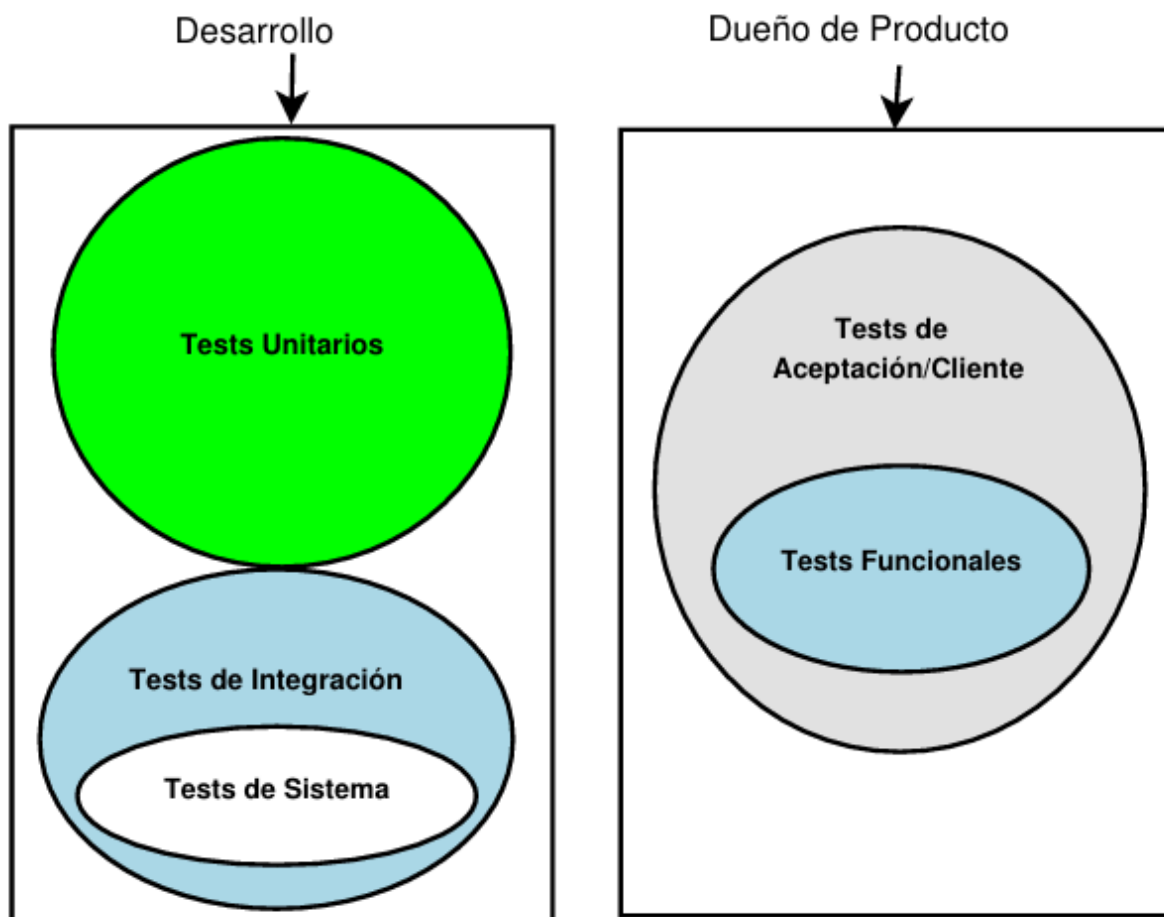
#### Anti-patrones Comunes

Pirámide Invertida: Demasiados tests de sistema  
Tests Frágiles: Se rompen con cambios mínimos  
Tests Lentos: La suite tarda horas  
Tests Acoplados: Dependen entre sí o del orden

**En TDD se usan principalmente estos tipos de pruebas:**

| Tipo de Test        | Propósito                                 | Ejemplo                                          |
|---------------------|-------------------------------------------|--------------------------------------------------|
| Tests de Aceptación | Validar requisitos de negocio             | "El usuario puede añadir un producto al carrito" |
| Tests Funcionales   | Verificar una funcionalidad concreta      | "El sistema calcula correctamente el IVA"        |
| Tests de Sistema    | Probar la aplicación completa desde la UI | "Flujo completo de compra en la web"             |

|                      |                                                                                                                              |                                                  |
|----------------------|------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| Tests Unitarios      | Probar una unidad mínima de código (clase, método)                                                                           | Calculadora.Sum(2,2)<br>== 4                     |
|                      | <ul style="list-style-type: none"> <li>- Atómico.</li> <li>- Independiente.</li> <li>- Inocuo.</li> <li>- Rápido.</li> </ul> |                                                  |
| Tests de Integración | Verificar la interacción entre componentes                                                                                   | "Capa de datos se conecta correctamente a la BD" |



### Características de un buen test Unitario (F.I.R.S.T)

- FAST: Rápido.
- Independent: Independiente de otros tests.
- Repeatable: Repetible.
- Self-Validating: Se autovalida.
- Timely: Escrito en el momento adecuado (antes del código).

“Si no sabes qué tipo de test estás escribiendo, quizás no tengas claro qué quieres probar.”

# Índice general

|           |                                                               |           |
|-----------|---------------------------------------------------------------|-----------|
| <b>I</b>  | <b>Base Teórica</b>                                           | <b>31</b> |
| <b>1.</b> | <b>El Agilismo</b>                                            | <b>33</b> |
| 1.1.      | Modelo en cascada . . . . .                                   | 35        |
| 1.2.      | Hablemos de cifras . . . . .                                  | 37        |
| 1.3.      | El manifiesto ágil . . . . .                                  | 38        |
| 1.4.      | ¿En qué consiste el agilismo?: Un enfoque práctico . . . . .  | 40        |
| 1.5.      | La situación actual . . . . .                                 | 44        |
| 1.6.      | Ágil parece, plátano es . . . . .                             | 46        |
| 1.7.      | Los roles dentro del equipo . . . . .                         | 47        |
| 1.8.      | ¿Por qué nos cuesta comenzar a ser ágiles? . . . . .          | 49        |
| <b>2.</b> | <b>¿Qué es el Desarrollo Dirigido por Tests? (TDD)</b>        | <b>53</b> |
| 2.1.      | El algoritmo TDD . . . . .                                    | 56        |
| 2.1.1.    | Escribir la especificación primero . . . . .                  | 56        |
| 2.1.2.    | Implementar el código que hace funcionar el ejemplo . . . . . | 57        |
| 2.1.3.    | Refactorizar . . . . .                                        | 58        |
| 2.2.      | Consideraciones y recomendaciones . . . . .                   | 60        |
| 2.2.1.    | Ventajas del desarrollador experto frente al junior . . . . . | 60        |
| 2.2.2.    | TDD con una tecnología desconocida . . . . .                  | 60        |
| 2.2.3.    | TDD en medio de un proyecto . . . . .                         | 61        |
| <b>3.</b> | <b>Desarrollo Dirigido por Tests de Aceptación (ATDD)</b>     | <b>63</b> |
| 3.1.      | Las historias de usuario . . . . .                            | 64        |
| 3.2.      | Qué y no Cómo . . . . .                                       | 68        |
| 3.3.      | ¿Está hecho o no? . . . . .                                   | 70        |
| 3.4.      | El contexto es esencial . . . . .                             | 71        |



|                                                                        |            |
|------------------------------------------------------------------------|------------|
| <b>4. Tipos de test y su importancia</b>                               | <b>73</b>  |
| 4.1. Terminología en la comunidad TDD . . . . .                        | 74         |
| 4.1.1. Tests de Aceptación . . . . .                                   | 75         |
| 4.1.2. Tests Funcionales . . . . .                                     | 76         |
| 4.1.3. Tests de Sistema . . . . .                                      | 76         |
| 4.1.4. Tests Unitarios . . . . .                                       | 78         |
| 4.1.5. Tests de Integración . . . . .                                  | 80         |
| <b>5. Tests unitarios y frameworks xUnit</b>                           | <b>83</b>  |
| 5.1. Las tres partes del test: AAA . . . . .                           | 84         |
| <b>6. Mocks y otros dobles de prueba</b>                               | <b>95</b>  |
| 6.1. Cuando usar un objeto real, un stub o un mock . . . . .           | 97         |
| 6.2. La metáfora Record/Replay . . . . .                               | 108        |
| <b>7. Diseño Orientado a Objetos</b>                                   | <b>111</b> |
| 7.1. Diseño Orientado a Objetos (OOD) . . . . .                        | 111        |
| 7.2. Principios S.O.L.I.D . . . . .                                    | 112        |
| 7.2.1. Single Responsibility Principle (SRP) . . . . .                 | 113        |
| 7.2.2. Open-Closed Principle (OCP) . . . . .                           | 113        |
| 7.2.3. Liskov Substitution Principle (LSP) . . . . .                   | 114        |
| 7.2.4. Interface Segregation Principle (ISP) . . . . .                 | 114        |
| 7.2.5. Dependency Inversión Principle (DIP) . . . . .                  | 115        |
| 7.3. Inversión del Control (IoC) . . . . .                             | 116        |
| <b>II Ejercicios Prácticos</b>                                         | <b>119</b> |
| <b>8. Inicio del proyecto - Test Unitarios</b>                         | <b>121</b> |
| <b>9. Continuación del proyecto - Test Unitarios</b>                   | <b>157</b> |
| <b>10. Fin del proyecto - Test de Integración</b>                      | <b>231</b> |
| 10.1. La frontera entre tests unitarios y tests de integración . . . . | 233        |
| 10.2. Diseño emergente con un ORM . . . . .                            | 243        |
| 10.2.1. Diseñando relaciones entre modelos . . . . .                   | 246        |
| 10.3. La unificación de las piezas del sistema . . . . .               | 247        |
| <b>11. La solución en versión Python</b>                               | <b>249</b> |
| <b>12. Antipatrones y Errores comunes</b>                              | <b>289</b> |

## Capítulo 2

# ¿Qué es el Desarrollo Dirigido por Tests? (TDD)

El Desarrollo Dirigido por Tests (Test Driven Development), al cual me referiré como **TDD**, es una técnica de diseño e implementación de software incluida dentro de la metodología XP. Coincido con Peter Provost<sup>1</sup> en que el nombre es un tanto desafortunado; algo como *Diseño Dirigido por Ejemplos* hubiese sido quizás mas apropiado. TDD es una técnica para diseñar software que se centra en tres pilares fundamentales:

- La implementación de las funciones justas que el cliente necesita y no más<sup>2</sup>.
- La minimización del número de defectos que llegan al software en fase de producción.
- La producción de software modular, altamente reutilizable y preparado para el cambio.

Cuando empezamos a leer sobre TDD creemos que se trata de una buena técnica para que nuestro código tenga una cobertura de tests muy alta, algo que siempre es deseable, pero es realmente una **herramienta de diseño que convierte al programador en un “oficial de primera”**. O, si no les gustan las metáforas, convierte al programador en desarrollador<sup>3</sup>. TDD es la respuesta a las grandes preguntas de:

---

<sup>1</sup><http://www.youtube.com/watch?v=JMEO6T6gkAA>

<sup>2</sup>Evitamos desarrollar funcionalidad que nunca será usada

<sup>3</sup>[http://www.ericssink.com/No\\_Programmers.html](http://www.ericssink.com/No_Programmers.html)

¿Cómo lo hago?, ¿Por dónde empiezo?, ¿Cómo sé qué es lo que hay que implementar y lo que no?, ¿Cómo escribir un código que se pueda modificar sin romper funcionalidad existente?

No se trata de escribir pruebas a granel como locos, sino de diseñar adecuadamente según los requisitos.

Pasamos, de pensar en implementar tareas, a pensar en ejemplos certeros que eliminen la ambigüedad creada por la prosa en lenguaje natural (nuestro idioma). Hasta ahora estábamos acostumbrados a que las tareas, o los casos de uso, eran las unidades de trabajo más pequeñas sobre las que ponerse a desarrollar código. Con TDD intentamos traducir el caso de uso o tarea en X ejemplos, hasta que el número de ejemplos sea suficiente como para describir la tarea sin lugar a malinterpretaciones de ningún tipo.

En otras metodologías de software, primero nos preocupamos de definir cómo va a ser nuestra arquitectura. Pensamos en las clases de infraestructura que van a homogeneizar la forma de trabajar en todos y cada uno de los casos, pensamos si vamos a usar un patrón Facade<sup>4</sup> y otro Singleton<sup>5</sup> y una comunicación mediante eventos, o DTOs, y una clase central que va a hacer esto y aquello... ¿Y si luego resulta que no necesitamos todo eso? ¿Cuánto vamos a tardar en darnos cuenta de ello? ¿Cuánto dinero vamos a malgastar? En TDD dejamos que la propia implementación de pequeños ejemplos, en constantes iteraciones, hagan emerger la arquitectura que necesitamos usar. Ni más ni menos. No es que nos despreocupemos por completo de las características técnicas de la aplicación a priori, es decir, lógicamente tendremos que saber si el desarrollo será para un teléfono móvil, para una web o para un pc de escritorio; más que nada porque tenemos que elegir unas herramientas de desarrollo conformes a las exigencias del guión. Sin embargo, nos limitamos a escoger el framework correspondiente y a usar su arquitectura como base. Por ejemplo, si escogiésemos Django<sup>6</sup> o ASP.NET MVC<sup>7</sup>, ya tendríamos definida buena parte de la base antes de empezar a escribir una sola línea de código. No es que trabajemos sin arquitectura, lógicamente, si en los requisitos está la interoperabilidad en las comunicaciones, tendremos que usar servicios web o servicios REST, lo cual ya propicia un determinado soporte. Lo que eliminamos son las arquitecturas encima de esas arquitecturas, las que intentan que todo se haga siempre igual y tal como se le ocurrió al “genio” de la empresa. A ser posible, esas que nos obligan a modificar siete ficheros para cambiar una cadena de texto. TDD produce una arquitectura que emerge de la no-ambigüedad de los tests automatizados,

<sup>4</sup>Facade: [http://es.wikipedia.org/wiki/Facade\\_\(patrón\\_de\\_diseño\)](http://es.wikipedia.org/wiki/Facade_(patrón_de_diseño))

<sup>5</sup>Singleton: <http://es.wikipedia.org/wiki/Singleton>

<sup>6</sup><http://www.djangoproject.com>

<sup>7</sup><http://www.asp.net/mvc/>

lo cual no exime de las revisiones de código entre compañeros ni de hacer preguntas a los desarrolladores más veteranos del equipo.

Las primeras páginas del libro de Kent Beck [3] (uno de los padres de la metodología XP) dan unos argumentos muy claros y directos sobre por qué merece la pena darle unos cuantos tragos a TDD, o mejor, por qué es beneficioso convertirla en nuestra herramienta de diseño principal. Estas son algunas de las razones que da Kent junto con otras destacadas figuras de la industria:

- La calidad del software aumenta (y veremos por qué).
- Conseguimos código altamente reutilizable.
- El trabajo en equipo se hace más fácil, une a las personas.
- Nos permite confiar en nuestros compañeros aunque tengan menos experiencia.
- Multiplica la comunicación entre los miembros del equipo.
- Las personas encargadas de la garantía de calidad adquieren un rol más inteligente e interesante.
- Escribir el ejemplo (test) antes que el código nos obliga a escribir el mínimo de funcionalidad necesaria, evitando sobrediseñar.
- Cuando revisamos un proyecto desarrollado mediante TDD, nos damos cuenta de que los tests son la mejor documentación técnica que podemos consultar a la hora de entender qué misión cumple cada pieza del puzzle.

Personalmente, añadiría lo siguiente:

- Incrementa la productividad.
- Nos hace descubrir y afrontar más casos de uso en tiempo de diseño.
- La jornada se hace mucho más amena.
- Uno se marcha a casa con la reconfortante sensación de que el trabajo está bien hecho.

Ahora bien, como cualquier técnica, no es una varita mágica y no dará el mismo resultado a un experto arquitecto de software que a un programador junior que está empezando. Sin embargo, es útil para ambos y para todo el rango de integrantes del equipo que hay entre uno y otro. Para el arquitecto es

su mano derecha, una guía que le hace clarificar el dominio de negocio a cada test y que le permite confiar en su equipo aunque tenga menos experiencia.

Frecuentemente, nos encontramos con gente muy desconfiada que mira con lupa el código de su equipo antes de que nadie pueda hacer “commit” al sistema de control de versiones. Esto se convierte en un cuello de botella porque hay varias personas esperando por el jefe (el arquitecto) para que dé el visto bueno y a este se le acumula el trabajo. Ni que decir tiene que el ambiente de trabajo en estos casos no es nada bueno ni productivo. Cuando el jefe sabe que su equipo hace TDD correctamente puede confiar en ellos y en lo que diga el sistema de integración continua<sup>8</sup> y las estadísticas del repositorio de código. Para el programador junior que no sabe por dónde va a coger al toro, o si es el toro quien le va a coger a él (o a ella), se convierte en el “Pepito Grillo” que le cuenta qué paso tiene que dar ahora. Y así, un paso tras otro, le guía en la implementación de la tarea que le ha sido asignada.

Cuando el equipo practica de esta manera, la comunicación fluye, la gente se vuelve más activa y la maquinaria funciona como un engranaje bien lubricado. Todos los que disfrutamos trabajando en el software llevamos dentro al personaje del buen arquitecto, entre muchos otros de nuestros personajes. La práctica que aquí se describe nos lo trae para que nos ayude y nos haga la vida más fácil.

## 2.1. El algoritmo TDD

La esencia de TDD es sencilla pero ponerla en práctica correctamente es cuestión de entrenamiento, como tantas otras cosas. El algoritmo TDD sólo tiene tres pasos:

- Escribir la especificación del requisito (el ejemplo, el test).
- Implementar el código según dicho ejemplo.
- Refactorizar para eliminar duplicidad y hacer mejoras.

Veámosla en detalle.

### 2.1.1. Escribir la especificación primero

Una vez que tenemos claro cuál es el requisito, lo expresamos en forma de código. Si estamos a nivel de aceptación o de historia, lo haremos con un framework tipo Fit, Fitnesse, Concordion o Cucumber. Esto es, ATDD. Si no, lo haremos con algún framework xUnit. ¿Cómo escribimos un test para

---

<sup>8</sup>[http://es.wikipedia.org/wiki/Continuous\\_integration](http://es.wikipedia.org/wiki/Continuous_integration)

un código que todavía no existe? Respondamos con otra pregunta ¿Acaso no es posible escribir una especificación antes de implementarla? Por citar un ejemplo conocido, las JSR (Java Specification Request) se escriben para que luego terceras partes las implementen... ¡ah, entonces es posible!. El framework Mono se ha implementado basándose en las especificaciones del ECMA-334 y ECMA-335 y funciona. Por eso, un test no es inicialmente un test sino un ejemplo o especificación. La palabra especificación podría tener la connotación de que es inamovible, algo preestablecido y fijo, pero no es así. Un test se puede modificar. Para poder escribirlo, tenemos que pensar primero en cómo queremos que sea la API del SUT<sup>9</sup>, es decir, tenemos que trazar antes de implementar. Pero sólo una parte pequeña, un comportamiento del SUT bien definido y sólo uno. Tenemos que hacer el esfuerzo de imaginar cómo sería el código del SUT si ya estuviera implementado y cómo comprobaríamos que, efectivamente, hace lo que le pedimos que haga. La diferencia con los que dictan una JSR es que no diseñamos todas las especificaciones antes de implementar cada una, sino que vamos una a una siguiendo los tres pasos del algoritmo TDD. El hecho de tener que usar una funcionalidad antes de haberla escrito le da un giro de 180 grados al código resultante. No vamos a empezar por fastidiarnos a nosotros mismos sino que nos cuidaremos de diseñar lo que nos sea más cómodo, más claro, siempre que cumpla con el requisito objetivo. En los próximos capítulos veremos cómo mediante ejemplos.

### 2.1.2. Implementar el código que hace funcionar el ejemplo

Teniendo el ejemplo escrito, codificamos lo mínimo necesario para que se cumpla, para que el test pase. Típicamente, el mínimo código es el de menor número de caracteres porque mínimo quiere decir el que menos tiempo nos llevó escribirlo. No importa que el código parezca feo o chapucero, eso lo vamos a enmendar en el siguiente paso y en las siguientes iteraciones. En este paso, la máxima es no implementar nada más que lo estrictamente obligatorio para cumplir la especificación actual. Y no se trata de hacerlo sin pensar, sino concentrados para ser eficientes. Parece fácil pero, al principio, no lo es; veremos que siempre escribimos más código del que hace falta. Si estamos bien concentrados, nos vendrán a la mente dudas sobre el comportamiento del SUT ante distintas entradas, es decir, los distintos flujos condicionales que pueden entrar en juego; el resto de especificaciones de este bloque de funcionalidad. Estaremos tentados de escribir el código que los gestiona sobre la marcha y, en ese momento, sólo la atención nos ayudará a contener el

<sup>9</sup>Subject Under Test. Es el objeto que nos ocupa, el que estamos diseñando a través de ejemplos.

impulso y a anotar las preguntas que nos han surgido en un lugar al margen para convertirlas en especificaciones que retomaremos después, en iteraciones consecutivas.

### 2.1.3. Refactorizar

Refactorizar no significa reescribir el código; reescribir es más general que refactorizar. Según Martín Fowler, refactorizar<sup>10</sup> es modificar el diseño sin alterar su comportamiento. A ser posible, sin alterar su API pública. En este tercer paso del algoritmo TDD, rastreamos el código (también el del test) en busca de líneas duplicadas y las eliminamos refactorizando. Además, revisamos que el código cumpla con ciertos principios de diseño (me inclino por S.O.L.I.D) y refactorizamos para que así sea. Siempre que llego al paso de refactorizar, y elimino la duplicidad, me planteo si el método en cuestión y su clase cumplen el Principio de una Única Responsabilidad<sup>11</sup> y demás principios.

El propio Fowler escribió uno de los libros más grandes de la literatura técnica moderna[7] en el que se describen las refactorizaciones más comunes. Cada una de ellas es como una receta de cocina. Dadas unas precondiciones, se aplican unos determinados cambios que mejoran el diseño del software mientras que su comportamiento sigue siendo el mismo. Mejora es una palabra ciertamente subjetiva, por lo que empleamos la métrica del código duplicado como parámetro de calidad. Si no existe código duplicado, entonces hemos conseguido uno de más calidad que el que presentaba duplicidad. Mas allá de la duplicidad, durante la refactorización podemos permitirnos darle una vuelta de tuerca al código para hacerlo más claro y fácil de mantener. Eso ya depende del conocimiento y la experiencia de cada uno. Los IDE como Eclipse, Netbeans o VisualStudio, son capaces de llevar a cabo las refactorizaciones más comunes. Basta con señalar un bloque de código y elegir la refactorización Extraer-Método, Extraer-Clase, Pull-up, Pull-down o cualquiera de las muchas disponibles. El IDE modifica el código por nosotros, asegurándonos que no se cometen errores en la transición. Al margen de estas refactorizaciones, existen otras más complejas que tienen que ver con la maestría del desarrollador y que a veces recuerdan al mago sacando un conejo de la chistera. Algunas de ellas tienen nombre y están catalogadas a modo de patrón y otras son anónimas pero igualmente eliminan la duplicidad. Cualquier cambio en los adentros del código, que mantenga su API pública, es una refactorización. La clave de una buena refactorización es hacerlo en pasitos muy pequeños. Se hace un cambio, se ejecutan todos los tests

<sup>10</sup><http://www.refactoring.com/>

<sup>11</sup>Ver Capítulo7 en la página 111

y, si todo sigue funcionando, se hace otro pequeño cambio. Cuando refactorizamos, pensamos en global, contemplamos la perspectiva general, pero actuamos en local. Es el momento de detectar malos olores y eliminarlos. El verbo refactorizar no existe como tal en la Real Academia Española pero, tras discutirlo en la red, nos resulta la mejor traducción del término refactoring. La tarea de buscar y eliminar código duplicado después de haber completado los dos pasos anteriores, es la que más tiende a olvidarse. Es común entrar en la dinámica de escribir el test, luego el SUT, y así sucesivamente olvidando la refactorización. Si de las tres etapas que tiene el algoritmo TDD dejamos atrás una, lógicamente no estamos practicando TDD sino otra cosa.

Otra forma de enumerar las tres fases del ciclo es:

- Rojo
- Verde
- Refactorizar

Es una descripción metafórica ya que los frameworks de tests suelen colorear en rojo aquellas especificaciones que no se cumplen y en verde las que lo hacen. Así, cuando escribimos el test, el primer color es rojo porque todavía no existe código que implemente el requisito. Una vez implementado, se pasa a verde.

Cuando hemos dado los tres pasos de la especificación que nos ocupa, tomamos la siguiente y volvemos a repetirlos. Parece demasiado simple, la reacción de los asistentes a mis cursos es mayoritariamente incrédula y es que el efecto TDD sólo se percibe cuando se practica. Me gusta decir que tiene una similitud con un buen vino; la primera vez que se prueba el vino en la vida, no gusta a nadie, pero a fuerza de repetir se convierte en un placer para los sentidos. Connotaciones alcohólicas a un lado, espero que se capte el mensaje.

¿Y TDD sirve para proyectos grandes? Un proyecto grande no es sino la agrupación de pequeños subproyectos y es ahora cuando toca aplicar aquello de "divide y vencerás". El tamaño del proyecto no guarda relación con la aplicabilidad de TDD. La clave está en saber dividir, en saber priorizar. De ahí la ayuda de Scrum para gestionar adecuadamente el *backlog* del producto. Por eso tanta gente combina XP y Scrum. Todavía no he encontrado ningún proyecto en el que se desaconseje aplicar TDD.

En la segunda parte del libro se expondrá el algoritmo TDD mediante ejemplos prácticos, donde iremos de menos a más, iterando progresivamente. No se preocupe si no lo ve del todo claro ahora.



## 2.2. Consideraciones y recomendaciones

### 2.2.1. Ventajas del desarrollador experto frente al junior

Existe la leyenda de que TDD únicamente es válido para personal altamente cualificado y con muchísima experiencia. Dista de la realidad; TDD es bueno para todos los individuos y en todos los proyectos. Eso sí, hay algunos matices. La diferencia entre el desarrollador experimentado que se sienta a hacer TDD y el junior, es cómo enfocan los tests, es decir, qué tests escriben; más allá del código que escriben. El experto en diseño orientado a objetos buscará un test que fuerce al SUT a tener una estructura o una API que sabe que le dará buenos resultados en términos de legibilidad y reusabilidad. Un experto es capaz de anticipar futuros casos de uso y futuros problemas y será más cuidadoso diseñando la API test tras test, aplicando las buenas prácticas que conoce. El junior probablemente se siente a escribir lo que mejor le parece, sin saber que la solución que elige quizás le traiga quebraderos de cabeza más adelante. La ventaja es que, cuando se dé cuenta de que su diseño tiene puntos a mejorar y empiece a refactorizar, contará con un importantísimo respaldo detrás en forma de batería de tests. Por poco experimentado que sea, se cuidará de no diseñar una API que le resulte casi imposible de usar. Debe tenerse en cuenta que se supone que el principiante no está solo, sino que en un contexto XP, hay desarrolladores de más experiencia que supervisarán y habrá momentos en los que se programe en parejas. La figura de los líderes es importante en XP al igual que en otras metodologías, con la gran diferencia de que el líder ágil está para responder preguntas y ayudar a los demás y no para darles látigo. El líder debe intentar que las personas que trabajan con él estén contentas de trabajar ahí y quieran seguir haciéndolo.

### 2.2.2. TDD con una tecnología desconocida

La primera vez que usamos una determinada tecnología o incluso una nueva librería, es complicado que podamos escribir la especificación antes que el SUT, porque no sabemos las limitaciones y fortalezas que ofrece la nueva herramienta. En estos casos, XP habla de *spikes* (disculpen que no lo traduzca, no sabría como). Un spike es un pequeño programa que se escribe para indagar en la herramienta, explorando su funcionalidad. Es hacerse alguna función o alguna aplicación pequeña que nos aporte el conocimiento que no tenemos. Si el spike es pequeño, y resulta que nos damos cuenta que su propio código es válido tal cual, entonces escribiremos el test justo a continuación, en lugar de dejarlo sin test. Sin un conocimiento básico de la API y las restricciones del sistema, no recomendaría lanzarse a escribir especificaciones. Hay que respetar el tiempo de aprendizaje con la herramienta y

avanzar una vez que tengamos confianza con ella. Intentar practicar TDD en un entorno desconocido es, a mi parecer, un antipatrón poco documentado. Tampoco es que descartemos forzosamente TDD, sino que primero tendremos que aprender a pilotar la máquina. Una vez sepamos si es de cambio manual o automático, dónde se encienden las luces y dónde se activa el limpia parabrisas, podremos echar a rodar. Es sólo cuestión de aplicar el sentido común, primero aprendemos a usar la herramienta y luego la usamos. Tenemos que evitar algo que pasa muy frecuentemente, minusvalorar el riesgo de no dominar las herramientas (y frameworks y lenguajes...)

### 2.2.3. TDD en medio de un proyecto

En la segunda parte del libro, la de los ejemplos prácticos, iniciamos el desarrollo de una aplicación desde cero. Igual que hacemos en los cursos que imparto. La pregunta de los asistentes aparece antes o después: ¿no se puede aplicar TDD en un proyecto que ya está parcialmente implementado? Claro que se puede, aunque con más consideraciones en juego. Para los nuevos requisitos de la aplicación, es decir, aquello que todavía falta por implementar, podremos aplicar eso de escribir el test primero y luego el código (¡y después refactorizar!). Es probable que el nuevo SUT colabore con partes legadas que no permiten la inyección de dependencias y que no cumplen una única responsabilidad<sup>12</sup>; código legado que nos dificulta su reutilización. El libro más recomendado por todos en los últimos tiempos sobre este asunto es, “Working Effectively with Legacy Code” de Michael C. Feathers[6]. Tratar con código legado no es moco de pavo. En general, por código legado entendemos que se trata de aquel que no tiene tests de ningún tipo. Mi recomendación, antes de ponerse a reescribir partes de código legado, es crear tests de sistema (y cuando el código lo permita, tests unitarios) que minimicen los posibles efectos colaterales de la reescritura. Si es una web, por ejemplo, agarrar Selenium o similar y grabar todos los posibles usos de la interfaz gráfica para poderlos reproducir después de las modificaciones y comprobar que todo el sistema se sigue comportando de la misma manera. Es un esfuerzo de usar y tirar porque estos tests son tremendamente frágiles, pero es mucho más seguro que lanzarse a reescribir alegremente. La siguiente recomendación es que la nueva API y la vieja convivan durante un tiempo, en lugar de reescribir eliminando la versión legada. Además de tener dos API podemos sobrecargar métodos para intentar que el código legado y su nueva versión convivan, si es que la API antigua nos sigue sirviendo. Viene siendo cuestión de aplicar el sentido común y recordar la ley de Murphy; “Si puede salir mal, saldrá mal”. Otra alternativa para hacer TDD con código nuevo

---

<sup>12</sup>Ver Capítulo 7 en la página 111

que colabora con código legado es abusar de los objetos mock<sup>13</sup>. Digamos que los tests van a ser más frágiles de lo que deberían pero es mejor usar paracaídas que saltar sin nada. Y por supuesto, si el nuevo código es más independiente, podemos seguir haciendo TDD sin ningún problema. Se lo recomiendo encarecidamente.

---

<sup>13</sup>Los veremos en el Capítulo 6 en la página 95

# Capítulo 3

## Desarrollo Dirigido por Tests de Aceptación (ATDD)

A pesar de la brevedad de este capítulo, puede considerarse probablemente *el más importante de todo el libro*. Si no somos capaces de entendernos con el cliente, ni la mejor técnica de desarrollo de todos los tiempos producirá un buen resultado.

La mayor diferencia entre las metodologías clásicas y la Programación Extrema es la forma en que se expresan los requisitos de negocio. En lugar de documentos de *word*, son ejemplos ejecutables. El Desarrollo Dirigido por Test de Aceptación (ATDD), técnica conocida también como Story Test-Driven Development (STDD), es igualmente TDD pero a un nivel diferente. Los tests de aceptación o de cliente son el criterio escrito de que el software cumple los requisitos de negocio que el cliente demanda. Son ejemplos escritos por los dueños de producto.

Es el punto de partida del desarrollo en cada iteración, la conexión perfecta entre Scrum y XP; allá donde una se queda y sigue la otra.

ATDD/STDD es una forma de afrontar la implementación de una manera totalmente distinta a las metodologías tradicionales. El trabajo del analista de negocio se transforma para reemplazar páginas y páginas de requisitos escritos en lenguaje natural (nuestro idioma), por ejemplos ejecutables surgidos del consenso entre los distintos miembros del equipo, incluido por supuesto el cliente. No hablo de reemplazar toda la documentación, sino los requisitos, los cuales considero un subconjunto de la documentación.

El algoritmo es el mismo de tres pasos pero son de mayor zancada que en el TDD practicado exclusivamente por desarrolladores. En ATDD la lista de ejemplos (tests) de cada historia, se escribe en una reunión que incluye a

dueños de producto, desarrolladores y responsables de calidad. Todo el equipo debe entender qué es lo que hay que hacer y por qué, para concretar el modo en que se certifica que el software lo hace. Como no hay única manera de decidir los criterios de aceptación, los distintos roles del equipo se apoyan entre sí para darles forma.

### 3.1. Las historias de usuario

Una historia de usuario posee similitudes con un caso de uso, salvando ciertas distancias. Por hacer una correspondencia entre historias de usuario y casos de uso, podríamos decir que el título de la historia se corresponde con el del caso de uso tradicional. Sin embargo, la historia no pretende definir el requisito. Escribir una definición formal incurre en el peligro de la imprecisión y la malinterpretación, mientras que contarlos con ejemplos ilustrativos, transmite la idea sin complicaciones. En ATDD cada historia de usuario contiene una lista de ejemplos que cuentan lo que el cliente quiere, con total claridad y ninguna ambigüedad. El enunciado de una historia es tan sólo una frase en lenguaje humano, de alrededor de cinco palabras, que resume qué es lo que hay que hacer. Ejemplos de historias podrían ser:

- *Formulario de inscripción*
- *Login en el sistema*
- *Reservar una habitación*
- *Añadir un libro al carrito de la compra*
- *Pago con tarjeta de crédito*
- *Anotar un día festivo en el calendario*
- *Informe de los artículos más vendidos*
- *Darse de baja en el foro*
- *Buscar casas de alquiler en Tenerife*

Breves, concretas y algo estimables. Son el resultado de escuchar al cliente y ayudarle a resumir el requisito en una sola frase. Muy importante: Están escritas con el vocabulario del negocio del cliente, no con vocabulario técnico. Por sí misma una historia aislada es difícil de estimar incluso con este formato. Lo que las hace estimables y nos hace ser capaces de estimarlas cada vez mejor, es el proceso evolutivo que llamamos ágil. Esto es: a base de iterar, estimar en cada iteración y hacer retrospectiva al final de la misma, vamos refinando la habilidad de escribir historias y estimarlas.

Canales[13] da una guía estupenda para las estimaciones entre las páginas 305 y 319 de su libro. Es más que recomendable leerlo. Sin embargo, desde la página 320 hasta la 343, discrepo con su forma de afrontar el análisis. Antes de conocer ATDD, también trabajaba como nos dice en esas páginas pero la experiencia me ha enseñado que no es la mejor manera. Saltar de casos de uso a crear un diagrama de clases modelando entidades, es en mi opinión, peligroso cuanto menos. Los diagramas nos pueden ayudar a observar el problema desde una perspectiva global, de manera que nos aproximamos al dominio del cliente de una manera más intuitiva. Pueden ayudarnos a comprender el dominio hasta que llegamos a ser capaces de formular ejemplos concretos. En cambio, representar elementos que formarán parte del código fuente mediante diagramas, es una fuente de problemas. Traducir diagramas en código fuente, es decir el modelado, es en cierto modo opuesto a lo que se expone en este libro. Para mí, la única utilidad que tiene el UML<sup>a</sup> es la de representar mediante un diagrama de clases, código fuente existente. Es decir, utilizo herramientas que autogeneran diagramas de clases, a partir de código, para poder echar un vistazo a las entidades de manera global pero nunca hago un diagrama de clases antes de programar. Mis entidades emergen a base de construir el código conforme a ejemplos. En todos los “ejemplos” que aparecen en las citadas páginas, realmente lo que leemos son descripciones, no son ejemplos potencialmente ejecutables. Definir entidades/modelos y hablar de pantallas antes de que haya una lista de ejemplos ejecutables y código ejecutable que los requiera, es un camino problemático. Como artesano del software, no creo en los generadores de aplicaciones.

---

<sup>a</sup>Lenguaje de Modelado Universal

Cada historia provoca una serie de preguntas acerca de los múltiples contextos en que se puede dar. Son las que naturalmente hacen los desarrolladores a los analistas de negocio o al cliente.

- ¿Qué hace el sistema si el libro que se quiere añadir al carrito ya está dentro de él?
- ¿Qué sucede si se ha agotado el libro en el almacén?
- ¿Se le indica al usuario que el libro ha sido añadido al carrito de la compra?

Las respuestas a estas preguntas son afirmaciones, ejemplos, los cuales transformamos en tests de aceptación. Por tanto, cada historia de usuario tiene asociados uno o varios tests de aceptación (ejemplos):

- *Cuando el libro X se añade al carrito, el sistema devuelve un mensaje que dice: ‘‘El libro X ha sido añadido al carrito’’*
- *Al mostrar el contenido del carrito aparece el libro X*
- *El libro X ya no aparece entre los libros a añadir al carrito*

Cuantas menos palabras para decir lo mismo, mejor:

- *Añadir libro X en stock produce: ‘‘El libro X ha sido añadido al carrito’’*
- *Libro X está contenido en el carrito*
- *Libro X ya no está en catálogo de libros*

Las preguntas surgidas de una historia de usuario pueden incluso dar lugar a otras historias que pasan a engrosar el backlog o lista de requisitos: “Si el libro no está en stock, se enviará un email al usuario cuando llegue”.

Los tests de aceptación son así; afirmaciones en lenguaje humano que tanto el cliente, como los desarrolladores, como la máquina, entienden. ¿La máquina? ¿cómo puede entender eso la máquina? Mágicamente no. El equipo de desarrollo tiene que hacer el esfuerzo de conectar esas frases con los puntos de entrada y salida del código. Para esto existen diversos frameworks libres y gratuitos que reducen el trabajo. Los más conocidos son FIT, Fitnesse, Concordion, Cucumber y Robot. Básicamente lo que proponen es escribir las frases con un formato determinado como por ejemplo HTML, usando etiquetas de una manera específica para delimitar qué partes de la frase son variables de entrada para el código y cuales son datos para validación del resultado de la ejecución. Como salida, Concordion por ejemplo, produce un HTML modificado que marca en rojo las afirmaciones que no se cumplieron, además de mostrar las estadísticas generales sobre cuántos tests pasaron y cuántos no. Veamos un ejemplo de la sintaxis de Concordion:

```
<html
 xmlns:concordion="http://www.concordion.org/2007/concordion">
<body>
 <p>
 El saludo para el usuario
 Manolo
 será:

 ¡Hola Manolo!
 </p>
</body>
</html>
```

Lógicamente, no le pedimos al cliente que se aprenda la sintaxis de Concordion y escriba el código HTML. Le pedimos que nos ayude a definir la frase o que nos la valide y luego, entre analistas de negocio, desarrolladores y testers (equipo de calidad), se escribirá el HTML. Lo interesante para el cliente es que el renderizado del HTML contiene el ejemplo que él entiende y es una bonita tarjeta que Concordion coloreará con ayuda de la hoja de estilos, subrayando en verde o en rojo según funcione el software. Concordion sabe dónde buscar la función `greetingsFor` y reconoce que el argumento con que la invocará es `Manolo`. Comparará el resultado de la ejecución con la frase `¡Hola Manolo!` y marcará el test como verde o rojo en función de ello. Un test de cliente o de aceptación con estos frameworks, a nivel de código, es un enlace entre el ejemplo y el código fuente que lo implementa. El propio framework se encarga de hacer la pregunta de si las afirmaciones son ciertas o no. Por tanto, su aspecto dista mucho de un test unitario o de integración con un framework `xUnit`.

Para cada test de aceptación de una historia de usuario, habrá un conjunto de tests unitarios y de integración de grano más fino que se encargará, primero, de ayudar a diseñar el software y, segundo, de afirmar que funciona como sus creadores querían que funcionase. Por eso ATDD o STDD es el comienzo del ciclo iterativo a nivel desarrollo, porque partiendo de un test de aceptación vamos profundizando en la implementación con sucesivos test unitarios hasta darle forma al código que finalmente cumple con el criterio de aceptación definido. No empezamos el diseño en torno a una supuesta interfaz gráfica de usuario ni con el diseño de unas tablas de base de datos, sino marcando unos criterios de aceptación que nos ayuden a ir desde el lado del negocio hasta el lado más técnico pero siempre concentrados en lo que el cliente demanda, ni más ni menos. Las ventajas son numerosas. En primer lugar, no trabajaremos en funciones que finalmente no se van a usar<sup>1</sup>.

---

<sup>1</sup>El famoso YAGNI (You ain't gonna need it)



En segundo lugar, forjaremos un código que está listo para cambiar si fuera necesario porque su diseño no está limitado por un diseño de base de datos ni por una interfaz de usuario. Es más fácil hacer modificaciones cuando se ha diseñado así, de arriba a abajo, en vez de abajo a arriba. Si el arquitecto diseña mal la estructura de un edificio, será muy complejo hacerle cambios en el futuro pero si pudiera ir montando la cocina y el salón sin necesidad de estructuras previas, para ir enseñándolas al cliente, seguro que al final colocaría la mejor de las estructuras para darle soporte a lo demás. En la construcción de viviendas eso no se puede hacer pero en el software, sí. Y, además, es lo natural, aunque estemos acostumbrados a lo contrario. ¡Porque una aplicación informática no es una casa!. Dada esta metáfora, se podría interpretar que deberíamos partir de una interfaz gráfica de usuario para la implementación pero no es cierto. Ver el dibujo de una interfaz gráfica de usuario no es como ver una cocina. Primero, porque la interfaz gráfica puede o no ser intuitiva, utilizable y, a consecuencia de esto, en segundo lugar, no es el medio adecuado para expresar qué es lo que el cliente necesita sino que la interfaz de usuario es parte del *cómo* se usa.

### 3.2. Qué y no Cómo

Una de las claves de ATDD es justamente que nos permite centrarnos en el **qué** y no en el **cómo**. Aprovechamos los frameworks tipo Concordion para desarrollar nuestra habilidad de preguntar al cliente qué quiere y no cómo lo quiere. Evitamos a toda costa ejemplos que se meten en el cómo hacer, más allá del qué hacer:

- *Al rellenar el cuadro de texto de buscar y pulsar el botón contiguo, los resultados aparecen en la tabla de la derecha*
- *Al introducir la fecha y hacer click en el botón de añadir, se crea un nuevo registro vacío*
- *Los libros se almacenan en la tabla Libro con campos: id, titulo y autor*
- *Seleccionar la opción de borrar del combo, marcar con un tick las líneas a borrar y verificar que se eliminan de la tabla al pulsar el botón aplicar.*
- *Aplicación Flash con destornilladores y tornillos girando en 3D para vender artículos de mi ferretería por Internet*

Cuando partimos de especificaciones como estas corremos el riesgo de pasar por alto el verdadero propósito de la aplicación, la información con auténtico

valor para el negocio del cliente. Salvo casos muy justificados, el Dueño del Producto no debe decir cómo se implementa su solución, igual que no le decimos al fontanero cómo tiene que colocar una tubería. La mayoría de las veces, el usuario no sabe exactamente lo que quiere pero, cuando le sugerimos ejemplos sin ambigüedad ni definiciones, generalmente sabe decirnos si es o no es eso lo que busca. Uno de los motivos por los que el cliente se empeña en pedir la solución de una determinada manera es porque se ha encontrado con profesionales poco experimentados que no le han sabido sugerir las formas adecuadas o que no llegaron a aportarle valor para su negocio. Con ATDD nos convertimos un poco en psicólogos en lugar de pretender ser adivinos. A base de colaboración encontramos y clasificamos la información que más beneficio genera para el usuario.

Encuentro particularmente difícil practicar ATDD cuando los dueños de producto están mal acostumbrados al sistema clásico en el que el análisis de los requisitos acaba produciendo un diagrama de componentes o módulos y luego un diagrama de clases. En las primeras reuniones de análisis, se empeñan en que dibujemos ese diagrama de módulos en los que el sistema se va a dividir a pesar de que les explique que eso no aporta más valor a su negocio. Les digo que la abstracción de los requisitos en forma de módulos o grupos no sirve más que para contaminar el software con falsos requisitos de negocio y para limitarnos a la hora de implementar, aunque a veces les resulta difícil de ver en un principio. Los únicos módulos que hay que identificar son los que tienen valor de negocio, es decir, aquellos conjuntos lógicos que tengan relación con una estrategia de negocio. Por ejemplo, de cara a ofrecer determinados servicios: servicio de venta, de alquiler, de consultoría...

La forma en que comprenden el proceso iterativo, es sentándose frente a ellos en un lugar cómodo y adoptando el rol de psicólogo de las películas norteamericanas, abordando los ejemplos. Una vez llevo la voz cantante, empiezo a formular ejemplos para que me digan si son válidos o no. Al principio no son capaces de distinguir entre una descripción y un ejemplo preciso por lo que se apresuran a darme descripciones que consideran suficientes como para implementar el software pero que para mí, ajeno a su negocio, no lo son:

- *Buscando por Santa Cruz de Tenerife, aparece una lista de pisos en alquiler.*

Entonces reconduzco la conversación haciéndoles ver que su descripción se corresponde en realidad con varios ejemplos.

- *Buscando que el precio sea inferior a 600€, e introduciendo el texto "Santa Cruz de Tenerife", el sistema muestra una lista de pisos que no superan los 600€ mensuales de alquiler y que se encuentran en la ciudad de Santa Cruz de Tenerife*
- *Buscando que el precio esté entre 500€ y 700€ y que tenga 2 habitaciones e introduciendo el texto "Santa Cruz de Tenerife", el sistema muestra una lista de pisos que cumplen las tres condiciones*
- *Buscando que tenga 3 habitaciones y 2 cuartos de baño, e introduciendo el texto "Santa Cruz de Tenerife", el sistema muestra una lista de pisos que cumplen las tres condiciones*
- *Buscando con el texto "Tenerife", el sistema muestra la lista de pisos de toda la provincia de Santa Cruz de Tenerife*
- *En la lista, cada piso se muestra mediante una fotografía y el número de habitaciones que tiene*

Para responder si los ejemplos son verdaderos o falsos, ellos mismos descubren dudas sobre lo que necesitan para su negocio. Dejan de ir teniendo pensamientos mágicos para ser conscientes de la precisión con que tenemos que definir el funcionamiento del sistema. A partir de ese momento, entienden que la distancia entre los expertos en desarrollo y los expertos en negocio va menguando y dejan de preocuparse por diagramas abstractos. Entonces dicen... "¿Tenemos que pensar todas estas cosas?" Y tengo que contarles que, aunque los ordenadores hayan avanzado mucho, no dejan de ser máquinas muy tontas. Les cuento que si esas decisiones sobre el negocio no me las validan ellos, tendré que decidir yo, que no soy experto en su negocio. Así comienzan a involucrarse más en el desarrollo y todos comenzamos a hablar el mismo idioma.

Al final, todo esto no consiste en otra cosa que en escribir ejemplos e implementarlos.

### 3.3. ¿Está hecho o no?

Otra ventaja de dirigir el desarrollo por las historias y, a su vez, por los ejemplos, es que vamos a poder comprobar muy rápido si el programa está cumpliendo los objetivos o no. Conocemos en qué punto estamos y cómo vamos progresando. El Dueño de Producto puede revisar los tests de aceptación y ver cuántos se están cumpliendo, así que nuestro trabajo gana una

confianza tremenda. Es una gran manera de fomentar una estrecha colaboración entre todos los roles el equipo. Piénselo bien: ¡la propia máquina es capaz de decirnos si el programa cumple las especificaciones el cliente o no!

De cara a hacer modificaciones en nuevas versiones del programa y lanzarlo a producción, el tiempo que tardamos en efectuar las pruebas de regresión disminuye de manera drástica, lo cual se traduce en un ahorro considerable.

Los tests de regresión deben su nombre al momento en que se ejecutan, no a su formato ni a otras características. Antes de lanzar una nueva versión de un producto en producción, ejecutamos todas las pruebas posibles, tanto manuales como automáticas para corroborar que tanto las nuevas funciones como las existentes funcionan. Regresión viene de regresar, puesto que regresamos a funcionalidad desarrollada en la versión anterior para validar que no se ha roto nada. Cuando no se dispone de una completa batería de tests, la regresión completa de una aplicación puede llevar varios días en los que el equipo de calidad ejecuta cada parte de la aplicación con todas y cada una de sus posibles variantes. Hablando en plata; una tortura y un gasto económico importante.

### 3.4. El contexto es esencial

Fuera del contexto ágil, ATDD tiene pocas probabilidades de éxito ya que si los analistas no trabajan estrechamente con los desarrolladores y testers, no se podrá originar un flujo de comunicación suficientemente rico como para que las preguntas y respuestas aporten valor al negocio. Si en lugar de ejemplos, se siguen escribiendo descripciones, estaremos aumentando la cantidad de trabajo considerablemente con lo cual el aumento de coste puede no retornar la inversión. Si los dueños de producto (cliente y analistas) no tienen tiempo para definir los tests de aceptación, no tiene sentido encargárselos a los desarrolladores, sería malgastar el dinero. Tener tiempo es un asunto muy relativo y muy delicado. No entraremos en ese tema tan escabroso al que yo llamaría más bien estar dispuestos a invertir el tiempo, más que tener o no tener tiempo. ¿Alguien tiene tiempo?. La herramienta con la que se escriben los tests de aceptación tiene que minimizar la cantidad de código que requiere esa conexión entre las frases y el código del sistema, si no, el mantenimiento se encarecerá demasiado. ATDD/STDD es un engranaje que cuesta poner en marcha pero que da sus frutos, como se puede leer en este artículo de la revista *Better Software* de 2004<sup>2</sup>.

<sup>2</sup><http://industriallogic.com/papers/storytest.pdf>

Desgraciadamente no podemos extendernos más con respecto a ATDD/STDD, si bien se podría escribir un libro sobre ello. Mike Cohn escribió uno muy popular titulado *User Stories Applied*[5] que le recomiendo encarecidamente leer. Mención especial también al capítulo sobre ATDD de Lasse Koskela en *Test Driven*[9] y los sucesivos, que incluyen ejemplos sobre el framework FIT. Gojko Adzic[1] tiene un libro basado en FitNesse y por supuesto cabe destacar su famoso libro sobre *Acceptance Testing*[2]. Elisabeth Hendrickson, en colaboración con otros expertos de la talla de Brian Marick, publicó un *paper* que puede leerse online<sup>3</sup> e incluye ejemplos en el framework Robot.

En la parte práctica de este libro tendremos ocasión de ver algunos ejemplos más aunque, por motivos de espacio, no es exhaustiva.

---

<sup>3</sup><http://testobsessed.com/wordpress/wp-content/uploads/2008/12/atddexample.pdf>

## Capítulo 4

# Tipos de test y su importancia

La nomenclatura sobre tests puede ser francamente caótica. Ha sido fuente de discusión en los últimos años y sigue sin existir universalmente de manera categórica. Cada equipo tiende a adoptar sus propias convenciones, ya que existen distintos aspectos a considerar para denominar tests. Por aspecto quiero decir que, según cómo se mire, el test se puede clasificar de una manera o de otra. Así se habla, por ejemplo, del aspecto visibilidad (si se sabe lo que hay dentro del SUT o no), del aspecto potestad (a quién pertenece el test), etc. Dale H. Emery hace una extensa recopilación de los posibles aspectos o puntos de vista y los tipos que alberga cada uno de ellos<sup>1</sup>. No es la única que existe, ni sus definiciones son extensas, pero nos da una idea de la complejidad del problema. Michael Feathers también enumera varias clasificaciones de tests en un post reciente<sup>2</sup>. Por su parte, la Wikipedia aporta otro punto de vista complementario a los anteriores<sup>3</sup>.

Definitivamente, cada comunidad usa unos términos diferentes. Una comunidad podría ser la de los que practicamos TDD, y otra podría ser la de los que escriben tests para aplicaciones ya implementadas que todavía no los tienen, por ejemplo. Un mismo test puede ser de varios tipos, incluso mirándolo desde la perspectiva de un solo equipo de desarrollo ya que su clasificación dependerá del aspecto a considerar. No existen reglas universales para escribir todos y cada uno de los tipos de tests ni sus posibles combinaciones. Es una cuestión que llega a ser artesanal. Sin embargo, más allá de los términos, es conveniente que tengamos una idea de cómo es cada tipo de test, según las convenciones que hayamos elegido, para ser coherentes a la hora de escribirlos. **Cada vez que vamos a programar un test, tenemos que**

<sup>1</sup><http://cwg.dhemery.com/2004/04/dimensions/>

<sup>2</sup><http://blog.objectmentor.com/articles/2009/04/13/x-tests-are-not-x-tests>

<sup>3</sup>[http://en.wikipedia.org/wiki/Software\\_testing](http://en.wikipedia.org/wiki/Software_testing)

estar seguros de por qué lo escribimos y de qué estamos probando. Es extremadamente importante tener claro qué queremos afirmar con cada test y por qué lo hacemos de esa manera. Podría ser que el hecho de no saber determinar qué tipo de test vamos a programar, indique que no estamos seguros de por qué lo escribimos. Dicho de otro modo, si no conseguimos darle nombre a lo que hacemos, quizás no sepamos por qué lo hacemos. Probablemente no tengamos claro el diseño que queremos, o puede que el test no esté probando lo que debe, o que no estemos delimitando responsabilidades adecuadamente, o quizás estemos escribiendo más tests de los que son necesarios. Es una heurística a tener en cuenta.

En el ámbito de TDD no hablamos de tests desde el aspecto visibilidad (típicamente tests de caja blanca y caja negra). Usamos otros términos, pero sabemos que un test de caja negra podría coincidir con un test unitario basado en el estado. Y un test de caja blanca podría coincidir con un test unitario basado en la interacción. No es una regla exacta porque, dependiendo de cómo se haya escrito el test, puede que no sea unitario sino de integración. Lo importante es ser capaces de entender la naturaleza de los tests.

En la siguiente sección veremos los términos más comunes dentro de la comunidad TDD y sus significados.

## 4.1. Terminología en la comunidad TDD

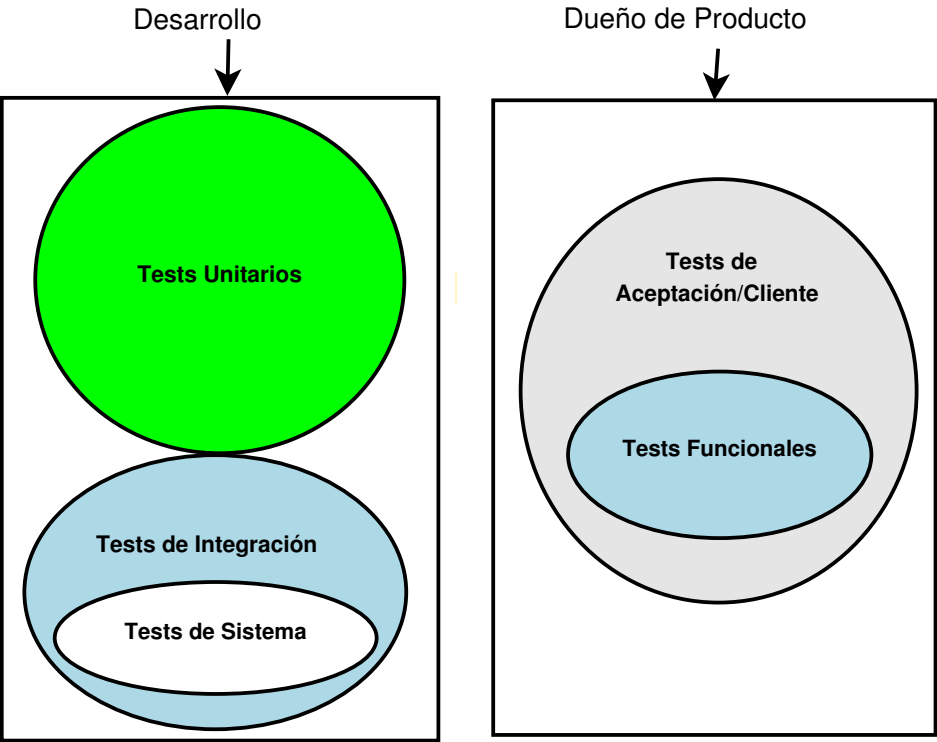
Desde el aspecto potestad, es decir, mirando los tests según a quién le pertenecen, distinguimos entre tests escritos por desarrolladores y tests escritos por el Dueño del Producto. Recordemos que el Dueño del Producto es el analista de negocio o bien el propio cliente. Lo ideal es que el analista de negocio ayude al cliente a escribir los tests para asegurarse de que las afirmaciones están totalmente libres de ambigüedad. Los tests que pertenecen al Dueño del Producto se llaman tests de cliente o de aceptación. Charlie Poole<sup>4</sup> prefiere llamarles tests de cliente ya que por aceptación se podría entender que se escriben al final cuando, realmente, no tiene que ser así. De hecho, en TDD partimos de tests de aceptación (ATDD) para conectar requerimientos con implementación, o sea, que los escribimos antes que nada. Cuando se escribe el código que permite ejecutar este test, y se ejecuta positivamente, se entiende que el cliente acepta el resultado. Por esto se habla de aceptación. Y también por esto es un término provocador, al haber clientes que niegan que un test de aceptación positivo signifique que aceptan esa parte del producto. Nosotros hablaremos de aceptación porque se usa más

---

<sup>4</sup>Uno de los padres de NUnit, con más de 30 años de experiencia: <http://www.charliepoole.org/cp.php>

en la literatura que test de cliente, aunque convendrá recordar lo peligrosa que puede llegar a ser esta denominación.

En el siguiente diagrama se muestra la clasificación de los tests típica de un entorno ATDD/TDD. A la izquierda, se agrupan los tests que pertenecen a desarrolladores y, a la derecha, los que pertenecen al Dueño del Producto. A su vez, algunos tipos de tests contienen a otros.



4.1.1. Tests de Aceptación

¿Cómo es un test de aceptación? Es un test que permite **comprobar** que el software **cumple con un requisito de negocio**. Como se vio en el capítulo de ATDD, un test de aceptación es un ejemplo escrito con el lenguaje del cliente pero que puede ser ejecutado por la máquina. Recordemos algunos ejemplos:



- *El producto X con precio 50€ tiene un precio final de 55€ después de aplicar el impuesto Z*
- *Si el paciente nació el 1 de junio de 1981, su edad es de 28 años en agosto de 2009*

¿Los tests de aceptación no usan la interfaz de usuario del programa? Podría ser que sí, pero en la mayoría de los casos la respuesta debe ser no. Los tests de carga y de rendimiento son de aceptación cuando el cliente los considera requisitos de negocio. Si el cliente no los requiere, serán tests de desarrollo.

#### 4.1.2. Tests Funcionales

Todos los tests son en realidad funcionales, puesto que todos ejercitan alguna función del SUT<sup>5</sup>, aunque en el nivel más elemental sea un método de una clase. No obstante, cuando se habla del aspecto funcional, se distingue entre test funcional y test no funcional. Un test funcional es un subconjunto de los tests de aceptación. Es decir, comprueban alguna funcionalidad con valor de negocio. Hasta ahora, todos los tests de aceptación que hemos visto son tests funcionales. Los tests de aceptación tienen un ámbito mayor porque hay requerimientos de negocio que hablan de tiempos de respuesta, capacidad de carga de la aplicación, etc; cuestiones que van más allá de la funcionalidad. Un test funcional es un test de aceptación pero, uno de aceptación, no tiene por qué ser funcional.

#### 4.1.3. Tests de Sistema

Es el mayor de los tests de integración, ya que integra varias partes del sistema. Se trata de un test que puede ir, incluso, de extremo a extremo de la aplicación o del sistema. Se habla de sistema porque es un término más general que aplicación, pero no se refiere a administración de sistemas, no es que estemos probando el servidor web o el servidor SMTP aunque, tales servicios, podrían ser una parte de nuestro sistema. Así pues, un test del sistema se ejercita tal cual lo haría el usuario humano, usando los mismos puntos de entrada (aquí sí es la interfaz gráfica) y llegando a modificar la base de datos o lo que haya en el otro extremo. ¿Cómo se puede automatizar el uso de la interfaz de usuario y validar que funciona? Hay software

<sup>5</sup>Subject Under Test; el código que estamos probando

que permite hacerlo. Por ejemplo, si la interfaz de usuario es web, el plugin Selenium<sup>6</sup> para el navegador Mozilla Firefox<sup>7</sup> nos permite registrar nuestra actividad en una página web como si estuviéramos grabando un vídeo para luego reproducir la secuencia automáticamente y detectar cambios en la respuesta del sitio web. Pongamos que grabo la forma en que relleno un formulario con una dirección de correo electrónico incorrecta para que el sitio web me envíe un mensaje de error. Cada vez que quiera volver a comprobar que el sitio web responde igual ante esa entrada, sólo tengo que ejecutar el test generado por Selenium. Hay herramientas que permiten hacer lo mismo mediante programación: nos dan una API para seleccionar controles gráficos, y accionarlos desde código fuente, comprobando el estado de la ejecución con sentencias condicionales o asertivas. El propio Selenium lo permite. Una de las herramientas más populares es Watir<sup>8</sup> para Ruby y sus versiones para otros lenguajes de programación (Watin para .Net). Para aplicaciones escritas con el framework Django (Python), se utiliza el cliente web<sup>9</sup>. Para aplicaciones de escritorio, hay frameworks específicos como UIAutomation<sup>10</sup> o NUnitForms<sup>11</sup> que también permiten manipular la interfaz gráfica desde código.

**Existen muchas formas de probar un sistema.** Supongamos que hemos implementado un servidor web ligero y queremos validar que, cada vez que alguien accede a una página, se registra su dirección ip en un fichero de registro (log). Podríamos hacer un script con algún comando que se conecte a una URL del servidor, al estilo de Wget<sup>12</sup> desde la misma máquina y después buscar la ip 127.0.0.1 en el fichero de log con Grep<sup>13</sup>. Sirva el ejemplo para recalcar que no hay una sola herramienta ni forma de escribir tests de sistema, más bien depende de cada sistema.

**Los tests de sistema son muy frágiles en el sentido de que cualquier cambio en cualquiera de las partes que componen el sistema, puede romperlos. No es recomendable escribir un gran número de ellos por su fragilidad. Si la cobertura de otros tipos de tests de granularidad más fina, como por ejemplo los unitarios, es amplia, la probabilidad de que los errores sólo se detecten con tests de sistema es muy baja. O sea, que si hemos ido haciendo TDD, no es productivo escribir tests de sistema para todas las posibles formas de uso del sistema, ya que esta redundancia se traduce en un aumento del costo**

<sup>6</sup><http://seleniumhq.org/projects/ide/>

<sup>7</sup><http://www.mozilla-europe.org/es/firefox/>

<sup>8</sup><http://wtr.rubyforge.org/>

<sup>9</sup>[http://www.djangoproject.com/documentation/models/test\\_client/](http://www.djangoproject.com/documentation/models/test_client/)

<sup>10</sup><http://msdn.microsoft.com/en-us/library/ms747327.aspx>

<sup>11</sup><http://nunitforms.sourceforge.net/>

<sup>12</sup>[http://es.wikipedia.org/wiki/GNU\\_Wget](http://es.wikipedia.org/wiki/GNU_Wget)

<sup>13</sup><http://es.wikipedia.org/wiki/Grep>

de mantenimiento de los tests. Por el contrario, si no tenemos escrito absolutamente ningún tipo de test, blindar la aplicación con tests de sistema será el paso más recomendable antes de hacer modificaciones en el código fuente. Luego, cuando ya hubiesen tests unitarios para los nuevos cambios introducidos, se podrían ir desechando tests de sistema.

¿Por qué se les llama tests de aceptación y tests funcionales a los tests de sistema? En mi opinión, es un error. Un test funcional es una frase escrita en lenguaje natural que utiliza el sistema para ejecutarse. En el caso de probar que una dirección de email es incorrecta, el test utilizará la parte del sistema que valida emails y devuelve mensajes de respuesta. Sin embargo, el requisito de negocio no debe entrar en cómo es el diseño de la interfaz de usuario. Por tanto, el test funcional no entraría a ejecutar el sistema desde el extremo de entrada que usa el usuario (la GUI), sino desde el que necesita para validar el requisito funcional. Si la mayoría de los criterios de aceptación se validan mediante tests funcionales, tan sólo nos harán falta unos pocos tests de sistema para comprobar que la GUI está bien conectada a la lógica de negocio. Esto hará que nuestros tests sean menos frágiles y estaremos alcanzando el mismo nivel de cobertura de posibles errores.

En la documentación de algunos frameworks, llaman test unitarios a tests que en verdad son de integración y, llaman tests funcionales, a tests que son de sistema. Llamar test funcional a un test de sistema no es un problema siempre que adoptemos esa convención en todo el equipo y todo el mundo sepa para qué es cada test.

En casos puntuales, un requisito de negocio podría involucrar la GUI, tal como pasa con el cliente de Gmail del iPhone por ejemplo. Está claro que el negocio en ese proyecto está directamente relacionado con la propia GUI. En ese caso, el test de sistema sería también un test funcional.

#### 4.1.4. Tests Unitarios

Son los tests más importantes para el practicante TDD, los ineludibles. Cada test unitario o test unidad (unit test en inglés) es un paso que andamos en el camino de la implementación del software. Todo test unitario debe ser:

- Atómico
- Independiente
- Inocuo
- Rápido

Si no cumple estas premisas entonces no es un test unitario, aunque se ejecute con una herramienta tipo xUnit.

Atómico significa que el test prueba la mínima cantidad de funcionalidad posible. Esto es, probará un solo comportamiento de un método de una clase. El mismo método puede presentar distintas respuestas ante distintas entradas o distinto contexto. El test unitario se ocupará exclusivamente de uno de esos comportamientos, es decir, de un único camino de ejecución. A veces, la llamada al método provoca que internamente se invoque a otros métodos; cuando esto ocurre, decimos que el test tiene menor granularidad, o que es menos fino. Lo ideal es que los tests unitarios ataquen a métodos lo más planos posibles, es decir, que prueben lo que es indivisible. La razón es que un test atómico nos evita tener que usar el depurador para encontrar un defecto en el SUT, puesto que su causa será muy evidente. Como veremos en la parte práctica, hay veces que vale la pena ser menos estrictos con la atomicidad del test, para evitar abusar de los dobles de prueba.

Independiente significa que un test no puede depender de otros para producir un resultado satisfactorio. No puede ser parte de una secuencia de tests que se deba ejecutar en un determinado orden. Debe funcionar siempre igual independientemente de que se ejecuten otros tests o no.

Inocuo significa que no altera el estado del sistema. Al ejecutarlo una vez, produce exactamente el mismo resultado que al ejecutarlo veinte veces. No altera la base de datos, ni envía emails ni crea ficheros, ni los borra. Es como si no se hubiera ejecutado.

Rápido tiene que ser porque ejecutamos un gran número de tests cada pocos minutos y se ha demostrado que tener que esperar unos cuantos segundos cada rato, resulta muy improductivo. Un sólo test tendría que ejecutarse en una pequeña fracción de segundo. La rapidez es tan importante que Kent Beck ha desarrollado recientemente una herramienta que ejecuta los tests desde el IDE Eclipse mientras escribimos código, para evitar dejar de trabajar en código mientras esperamos por el resultado de la ejecución. Se llama JUnit Max<sup>14</sup>. Olof Bjarnason ha escrito otra similar y libre para Python<sup>15</sup>.

Para conseguir cumplir estos requisitos, un test unitario aísla la parte del SUT que necesita ejercitar de tal manera que el resto está inactivo durante la ejecución. Hay principalmente dos formas de validar el resultado de la ejecución del test: validación del estado y validación de la interacción, o del comportamiento. En los siguientes capítulos los veremos en detalle con ejemplos de código.

<sup>14</sup><http://www.junitmax.com/junitmax/subscribe.html>

<sup>15</sup><https://code.launchpad.net/objarni/+junk/pytddmon>

Los desarrolladores utilizamos los tests unitarios para asegurarnos de que el código funciona como esperamos que funcione, al igual que el cliente usa los tests de cliente para asegurarse que los requisitos de negocio se alcancen como se espera que lo hagan.

### **F.I.R.S.T**

Como los acrónimos no dejan de estar de moda, cabe destacar que las características de los tests unitarios también se agrupan bajo las siglas F.I.R.S.T que vienen de: Fast, Independent, Repeatable, Small y Transparent. Repetible encaja con inocuo, pequeño caza con atómico y transparente quiere decir que el test debe comunicar perfectamente la intención del autor.

#### **4.1.5. Tests de Integración**

Por último, los tests de integración son la pieza del puzzle que nos faltaba para cubrir el hueco entre los tests unitarios y los de sistema. Los tests de integración se pueden ver como tests de sistema pequeños. Típicamente, también se escriben usando herramientas xUnit y tienen un aspecto parecido a los tests unitarios, sólo que estos pueden romper las reglas. Como su nombre indica, integración significa que ayuda a unir distintas partes del sistema. Un test de integración puede escribir y leer de base de datos para comprobar que, efectivamente, la lógica de negocio entiende datos reales. Es el complemento a los tests unitarios, donde habíamos “falseado” el acceso a datos para limitarnos a trabajar con la lógica de manera aislada. Un test de integración podría ser aquel que ejecuta la capa de negocio y después consulta la base de datos para afirmar que todo el proceso, desde negocio hacia abajo, fue bien. Son, por tanto, de granularidad más gruesa y más frágiles que los tests unitarios, con lo que el número de tests de integración tiende a ser menor que el número de tests unitarios. Una vez que se ha probado que dos módulos, objetos o capas se integran bien, no es necesario repetir el test para otra variante de la lógica de negocio; para eso habrán varios tests unitarios. Aunque los tests de integración pueden saltarse las reglas, por motivos de productividad es conveniente que traten de ser inocuos y rápidos. Si tiene que acceder a base de datos, es conveniente que luego la deje como estaba. Por eso, algunos frameworks para Ruby y Python entre otros, tienen la capacidad de crear una base de datos temporal antes de ejecutar la batería de tests, que se destruye al terminar las pruebas. Como se trata de una herramienta incorporada, también hay quien ejecuta los tests unitarios creando y destruyendo bases de datos temporales pero es una práctica que debe evitarse porque los segundos extra que se necesitan para eso nos hacen